

No Memory Abstraction

1 What This Topic Means

Core Definition

No memory abstraction means a program uses **physical memory addresses** directly. The address written in the instruction is the actual address used by the hardware.

The simplest possible memory model is this:

$$\text{program address} = \text{physical memory address}$$

Memory is viewed as a linear sequence of addresses:

$$0, 1, 2, 3, \dots, \text{maximum address.}$$

Each address names a memory cell, usually one byte. If a program executes the instruction below, the CPU reads physical memory address 1000.

```
MOV REGISTER1,1000
```

There is no private address space, no virtual memory, and no automatic boundary between one program and another.

Historical Context

This model was used by early mainframes, early minicomputers, and early personal computers. It still appears in simple embedded systems where all software is known in advance.

2 Why Memory Abstraction Is Needed

The Problem

If two independent programs use the same physical address, one program can overwrite the other's code or data.

Suppose two programs are loaded at the same time:

- Program P_1 stores its variable `count` at physical address 2000.
- Program P_2 stores an instruction or variable at physical address 2000.

If P_1 writes a new value to address 2000, it changes the memory cell that P_2 is also using. The result may be corrupted data, a wrong instruction, or an immediate crash.

Main Danger

No memory abstraction means no isolation. A bug in a user program can damage another program or the operating system itself.

3 Three Simple Memory Organizations

Even in a system with no memory abstraction, the operating system must live somewhere in memory. Three simple layouts are common historically.

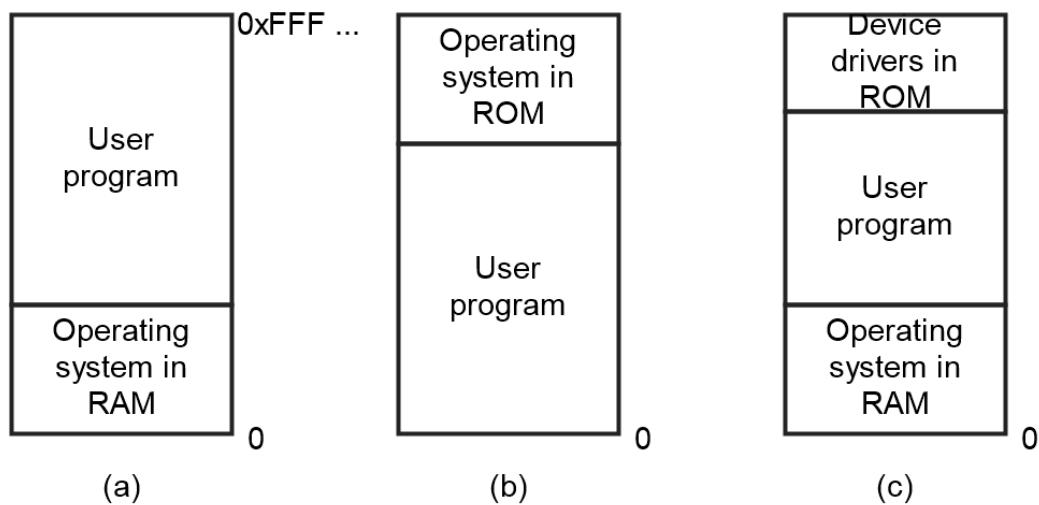


Figure 3-1. Three simple ways of organizing memory with an operating system and one user process. Other possibilities also exist.

Layout	Explanation
(a) OS in RAM at bottom	The user program is placed above the operating system. This was used historically on some mainframes and minicomputers, but a bad user program can overwrite the OS.
(b) OS in ROM at top	Operating system code is stored in read-only memory. This is safer for fixed systems because ROM cannot be overwritten by normal stores.
(c) BIOS-style split	Device drivers are in ROM, and the rest of the OS is in RAM. This was used by early PCs; the ROM part is commonly called the BIOS.

Key Observation

Layouts (a) and (c) are vulnerable because at least part of the operating system is writable RAM.

4 How a One-Program System Runs

When the machine can safely hold only one user program at a time, execution follows a simple cycle.

Step	Action
1	The user types a command.
2	The operating system loads the requested program from disk into memory.
3	The program runs using physical memory addresses.
4	The program finishes and returns control to the operating system.
5	The operating system displays a prompt and waits for the next command.
6	The next program is loaded, overwriting the previous program.

Single-Program Rule

At most one unrelated user program should occupy memory at a time.

5 Can Threads Help?

Threads can provide limited parallelism because all threads in one process are meant to share the same memory image.

Case	Effect in a no-abstraction system
Multiple threads in one process	Acceptable in principle because they intentionally share memory.
Multiple unrelated programs	Unsafe because each program expects its own independent memory.

Why Threads Are Not Enough

Threads do not give each program a private memory space. They only allow parallel activity inside one shared memory image.

Also, a system primitive enough to lack memory abstraction is unlikely to provide a rich thread abstraction.

6 Running Multiple Programs Anyway

Even without a true memory abstraction, a system can attempt to run multiple programs by using extra mechanisms.

6.1 Method 1: Swapping

Swapping

Swapping saves the entire memory image of one program to disk, then loads another program into memory.

The important point is that only one program is physically present in memory at a time. This avoids direct memory conflicts, but it is expensive because disk I/O is much slower than memory access.

Advantage	Disadvantage
Prevents two programs from overwriting each other in memory.	Saving and restoring all of memory is slow.
Does not require each program to have a private address space.	Only one program actually runs from memory at a time.

6.2 Method 2: Protection Keys

Some machines added hardware protection while still using physical addresses. Early IBM 360 systems used this approach.

IBM 360 Protection Keys

Memory was divided into 2-KB blocks. Each block had a 4-bit protection key, and the running process had a 4-bit key in the program status word (PSW).

Access Rule

$$\text{access allowed} \iff \text{PSW key} = \text{memory-block key}$$

If the keys did not match, the hardware generated a trap. Since only the operating system could change protection keys, user programs could be prevented from damaging one another.

Storage Needed for Protection Keys

For a 1-MB memory divided into 2-KB blocks:

$$1 \text{ MB} = 1024 \text{ KB}$$

$$\frac{1024 \text{ KB}}{2 \text{ KB/block}} = 512 \text{ blocks}$$

Each block needs a 4-bit key:

$$512 \times 4 = 2048 \text{ bits}$$

Since 8 bits make 1 byte:

$$\frac{2048}{8} = 256 \text{ bytes}$$

Answer

A 1-MB IBM 360-style memory needs only 256 bytes of protection-key storage.

7 Why Protection Alone Is Not Enough

Protection keys solve one problem: they stop a program from accessing memory blocks with the wrong key. However, they do not solve the deeper addressing problem.

Relocation Problem

The **relocation problem** occurs when a program contains absolute addresses that are correct only if the program is loaded at a specific physical location.

Consider two programs, each 16 KB in size.

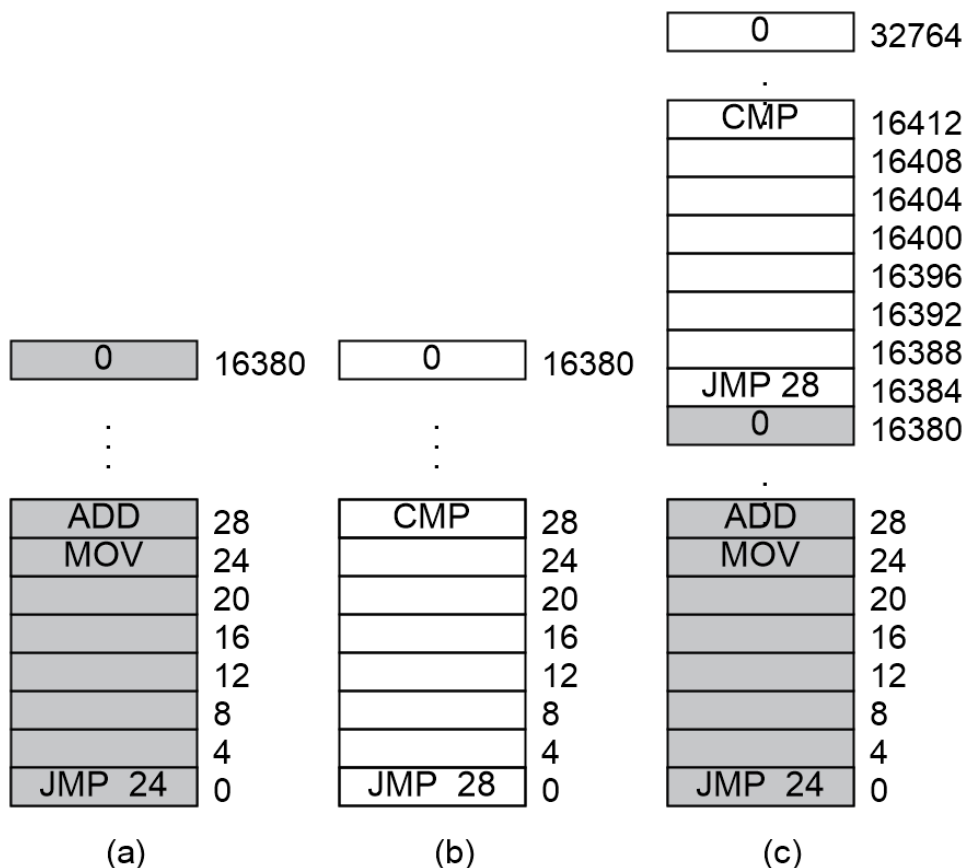


Figure 3-2. Illustration of the relocation problem. (a) A 16-KB program. (b) Another 16-KB program. (c) The two programs loaded consecutively into memory.

Program *A* is loaded first, beginning at physical address 0. Since Program *A* is 16 KB in size, Program *B* is loaded immediately after it, beginning at physical address 16,384.

Local Address vs Physical Address

Program *B* was written as if its first instruction were at address 0. So, inside Program *B*, address 28 means “the instruction 28 bytes from the start of Program *B*.”

But the hardware has no such idea of “inside Program *B*.” Without memory abstraction, address 28 means physical memory address 28.

Program *B*’s first instruction is:

```
JMP 28
```

The programmer intended this instruction to jump to Program *B*’s local address 28, where the `CMP` instruction is stored. Since Program *B* actually starts at physical address 16,384, Program *B*’s local address 28 is physically located at:

physical location of Program *B*'s local address 28 = 16,384 + 28 = 16,412.

However, the CPU does not automatically add 16,384. It executes the address exactly as written:

View	Meaning of JMP 28
Programmer's intention	Jump to Program <i>B</i> 's own local address 28, which is physical address 16,412.
CPU's interpretation	Jump directly to physical address 28, because no relocation or virtual memory is being used.
Actual result	Execution enters Program <i>A</i> 's memory instead of Program <i>B</i> 's CMP instruction.

Core Failure

The failure is not just lack of protection. The deeper problem is that absolute addresses are not automatically adjusted when a program is loaded somewhere other than address 0. Protection keys may stop some illegal accesses, but they do not make address 28 local to Program *B*.

8 Static Relocation

Definition

Static relocation fixes absolute addresses while the program is being loaded into memory.

If the loader places a program at base address *B*, then each relocatable address *A* inside the program is changed to:

Static Relocation Formula

$$\text{relocated physical address} = B + A$$

For Program *B*, the base address is 16,384. Therefore:

$$\text{JMP } 28 \Rightarrow \text{JMP } (16,384 + 28) \Rightarrow \text{JMP } 16412$$

Why This Helps

The program can be loaded somewhere other than address 0, and its internal address references can still point to the intended physical locations.

8.1 Limitations of Static Relocation

Static relocation is only a partial solution.

Limitation	Explanation
Slow loading	The loader must scan and modify address fields before execution.
Extra metadata	The executable must mark which words are relocatable addresses.
Constants are ambiguous	A number may be an address in one instruction but ordinary data in another.
Not flexible at runtime	Once loaded and patched, moving the program again is difficult.

The loader must relocate the 28 in:

```
JMP 28
```

But it must not relocate the 28 in:

```
MOV REGISTER1,28
```

because the second instruction may mean “put the constant value 28 into REGISTER1.”

9 Modern Relevance

Direct addressing of physical memory is no longer normal on general-purpose computers such as desktops, laptops, servers, and smartphones. These systems use stronger memory abstractions, usually including virtual address spaces and hardware memory management.

However, no memory abstraction is still practical in small embedded systems.

- **Microwave oven firmware:** the program is fixed and stored in ROM.
- **Washing machine controller:** users do not install unrelated applications.
- **Simple radio firmware:** memory use is known during design.
- **Smart card system:** the system is small and tightly controlled.

Some embedded operating systems are closer to libraries than full operating systems. The application links with the OS library and calls routines for I/O and common services. eCos is an example of this style.

10 Comparison of Approaches

Approach	Evaluation
One program at a time	Very simple, but it gives no real multiprogramming. A new program overwrites the old one.
Threads	Useful only inside one shared process image. It does not support unrelated protected programs.
Swapping	Avoids simultaneous memory conflicts by keeping one program in memory, but disk copying is slow.
Protection keys	Blocks many illegal accesses, but absolute addresses still refer to physical memory.
Static relocation	Lets programs load at different physical bases, but it needs relocation metadata and slows loading.

Address Spaces

1 Core Idea

Definition

An **address space** is the set of memory addresses that a process is allowed to use.

The address space abstraction gives each process the illusion of private memory.

Big Idea

process address $\neq$ physical address

Why It Matters

Address spaces solve two problems that direct physical addressing cannot handle well: **protection** and **relocation**.

2 Problem With Physical Memory

If programs can use physical memory directly, several things can go wrong.

Problem	Result
User program can address all memory	It may overwrite the operating system.
Two programs use the same address	They may overwrite each other's code or data.
Program loaded in a different place	Absolute jumps such as <code>JMP 28</code> may go to the wrong physical address.

Main Issue

Physical memory exposes real machine addresses. Address spaces hide those addresses behind a safer process-level view.

3 Protection and Relocation

Need	Question answered
Protection	Is this process allowed to access this address?
Relocation	Which physical address should this process address map to?

Protection alone is not enough. IBM 360-style protection keys could stop illegal access, but they did not automatically turn Program *B*'s address 28 into the correct physical address 16,412.

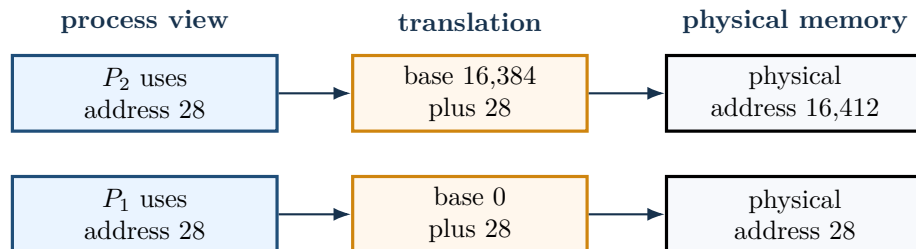
4 Address Space Examples

An address space is any set of valid addresses or names.

System	Address space
Process memory	Addresses a process may use.
Telephone numbers	All valid phone numbers in a numbering system.
x86 I/O ports	0 to 16,383.
IPv4	0 to $2^{32} - 1$, with reserved ranges.
.com domains	Valid domain-name strings ending in .com.

5 Private Address Spaces

Each process can use the same address values, but those values can map to different physical memory locations.



Interpretation

The address value 28 is not globally fixed. Its physical meaning depends on which process is running.

6 Base and Limit Registers

Mechanism

A simple way to implement address spaces is to use two CPU registers: **base** and **limit**.

Register	Meaning
Base	Starting physical address of the process.
Limit	Size of the process's legal address range.

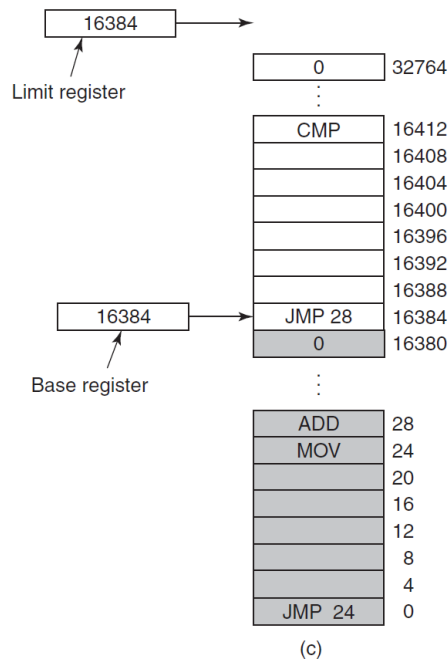


Figure 3-3. Base and limit registers can be used to give each process a separate address space.

Figure Interpretation

For the second program, the base register contains 16,384. A local address such as 28 is translated to physical address $16,384 + 28 = 16,412$. The limit register checks that the local address is inside the process's allowed range.

When the operating system runs a process, it loads that process's base and limit values into the CPU. User programs normally cannot change these registers.

7 Translation Rule

Let:

- A = address generated by the process;
- B = base register value;
- L = limit register value.

Base-Limit Translation

$$\begin{aligned} A < L &\Rightarrow \text{physical address} = B + A \\ A \geq L &\Rightarrow \text{fault} \end{aligned}$$

Two Jobs at Once

The limit check gives protection. The base addition gives relocation.

8 Translation Examples

Setup

Program B starts at physical address 16,384 and has length 16 KB.

$$B = 16,384, \quad L = 16,384$$

Legal address: JMP 28

Program B executes:

JMP 28

Step	Result
Generated address	$A = 28$
Limit check	$28 < 16,384$, so the access is legal.
Translation	$16,384 + 28 = 16,412$

Result

JMP 28 lands at physical address 16,412, which is Program B 's own instruction.

Illegal address: $A = 20,000$

Step	Result
Generated address	$A = 20,000$
Limit check	$20,000 \geq 16,384$, so the address is outside the process.
Translation	No physical address is produced. The CPU raises a fault.

Result

The access is aborted before memory is touched.

9 Dynamic Relocation

Definition

Dynamic relocation means addresses are translated while the program runs.

This avoids changing the program file while loading. The hardware adds the base register on every instruction fetch and data access.

Method	Difference
Static relocation	Loader modifies address constants before execution.
Dynamic relocation	CPU translates addresses during execution.

10 Strengths and Weaknesses

Point	Explanation
Strength	Simple private address spaces.
Strength	Protects against out-of-range memory access.
Strength	Program can be loaded wherever there is room.
Weakness	Every memory reference needs one comparison and one addition.
Weakness	Each process usually needs one contiguous block of physical memory.

Hardware Cost

Base-limit relocation is simple, but the CPU must check and translate every memory reference.

11 Quick Comparison

Concept	Meaning
Physical memory only	Program addresses are real physical addresses.
Address space	Process has its own usable address range.
Base register	Where the process starts in physical memory.
Limit register	How far the process may address.
Protection	Prevent access outside the process.
Relocation	Convert process address to physical address.

Swapping

1 Core Idea

Definition

Swapping is a memory-management technique where the operating system moves an entire process between main memory and disk.

When RAM is not large enough to hold all active processes, the operating system keeps some processes in memory and stores idle processes on disk.

Main Purpose

Swapping lets the system run more processes than can fit in RAM at the same time.

2 Why Swapping Is Needed

Modern systems often start many processes automatically. Some check for updates, incoming mail, network connections, or background services. Large applications can also need hundreds of megabytes or many gigabytes.

Situation	Memory pressure
Many background processes	RAM is consumed before the user opens an application.
Large applications	Programs such as image editors may need hundreds of MB or more.
Limited physical memory	Not every process can stay in RAM permanently.

3 Two Approaches to Memory Overload

Approach	Main idea
Swapping	Move a whole process into memory, run it, then move the whole process back to disk.
Virtual memory	Run a process even when only part of it is in main memory.

This Note

Swapping works at the level of whole processes. Virtual memory works at a finer level and is studied separately.

4 How Swapping Works

The basic cycle is simple:

1. Bring a process from disk into RAM.
2. Run it for a while.
3. If memory is needed, move an idle process back to disk.
4. Load another process into the freed memory.

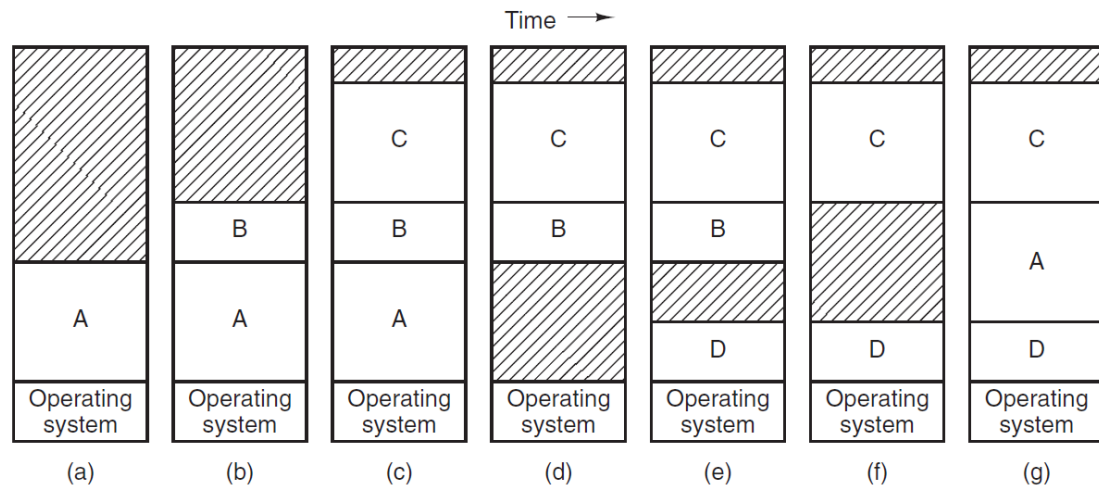


Figure 3-4. Memory allocation changes as processes come into memory and leave it. The shaded regions are unused memory.

Stage	Meaning
(a)	Only process <i>A</i> is in memory.
(b), (c)	Processes <i>B</i> and <i>C</i> are brought into memory.
(d)	Process <i>A</i> is swapped out, leaving a hole.
(e), (f)	Process <i>D</i> is loaded; later <i>B</i> is swapped out.
(g)	Process <i>A</i> returns, possibly at a different memory location.

5 Relocation After Swapping

When a process is swapped back in, it may not return to the same physical address.

Important Consequence

If a process moves to a different physical location, its addresses must still work correctly.

This can be handled in two ways:

Method	Explanation
Software relocation	Adjust addresses when the process is loaded.
Hardware relocation	Use base and limit registers during execution.

Why Base and Limit Help

The base register can point to the process's current physical starting address. If the process moves, the operating system updates the base register.

6 Holes and Memory Compaction

Swapping creates **holes**: unused regions between processes.

Memory Compaction

Memory compaction moves processes downward in memory so many small holes become one large hole.

Idea	Cost
Move processes together	Creates one large free region.
Copy large parts of memory	Uses a lot of CPU time and memory bandwidth.

Why Compaction Is Rare

Compaction can be expensive. Copying many gigabytes of RAM takes noticeable time, so systems avoid doing it frequently.

7 Allocating Space for Growth

If process size never changes, allocation is easy: give the process exactly the memory it needs.

Real processes often grow:

- the heap grows when the process dynamically allocates memory;
- the stack grows when function calls, local variables, and return addresses are added.

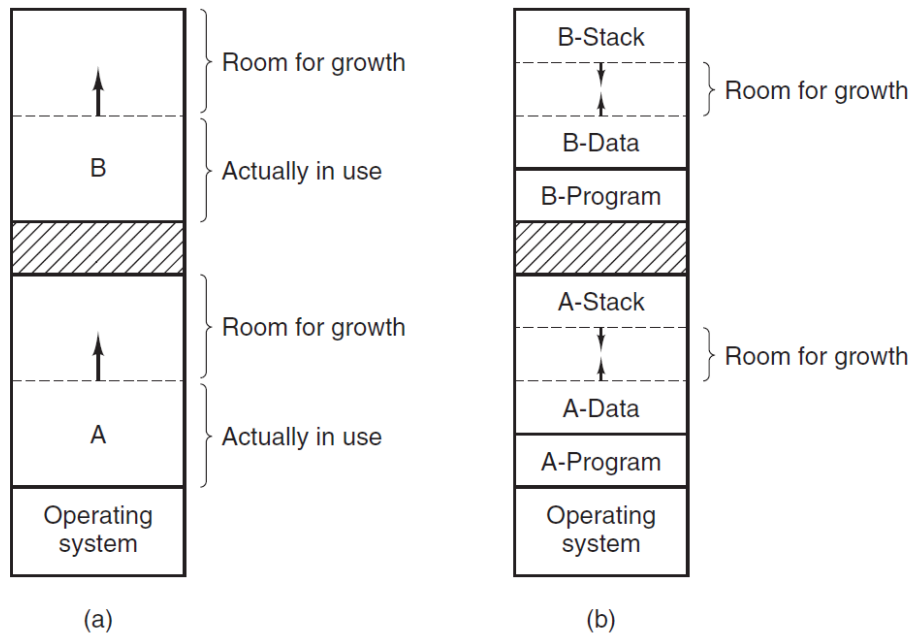


Figure 3-5. (a) Allocating space for a growing data segment. (b) Allocating space for a growing stack and a growing data segment.

8 Growing Data Segment

In Fig. 3-5(a), extra space is reserved above each process.

Part	Meaning
Actually in use	Memory currently needed by the process.
Room for growth	Extra space reserved so the process can grow without being moved immediately.
Hole between processes	Unused memory that may be allocated later.

Benefit

Allocating extra room reduces how often a growing process must be moved or swapped out.

9 Growing Stack and Heap

In Fig. 3-5(b), each process has:

- program text at the bottom;
- data or heap above the program text, growing upward;
- stack at the top, growing downward;
- free space between heap and stack.

Growth Direction

heap grows upward and stack grows downward

The free space between them can be used by either segment. If the space runs out, the operating system must take action.

10 What Happens When a Process Cannot Grow?

Case	OS action
Adjacent hole exists	Allow the process to grow into the hole.
No adjacent hole	Move the process to a larger hole, if one exists.
No large enough hole	Swap out one or more processes to create space.
No RAM and swap space full	Suspend the process until space is available, or kill it.

Do Not Swap Extra Space

When a process is swapped to disk, only memory actually in use should be written out. Swapping unused growth space wastes disk I/O.

11 Swapping Summary

Point	Takeaway
Unit moved	Whole process.
Main benefit	More processes can exist than fit in RAM at once.
Main cost	Disk I/O is slow.
Main side effect	Holes appear in memory.
Compaction	Combines holes, but costs CPU time.
Growth space	Reduces movement, but unused space should not be swapped.

Managing Free Memory

1 Core Idea

Definition

Free-memory management is how the operating system records which parts of memory are free and which parts are occupied.

When memory is assigned dynamically, processes enter, leave, grow, and shrink. The operating system must quickly answer two questions:

- Which memory regions are free?
- Where can a new or swapped-in process fit?

Two Main Methods

Free memory is commonly tracked using **bitmaps** or **linked lists**.

2 Memory Picture

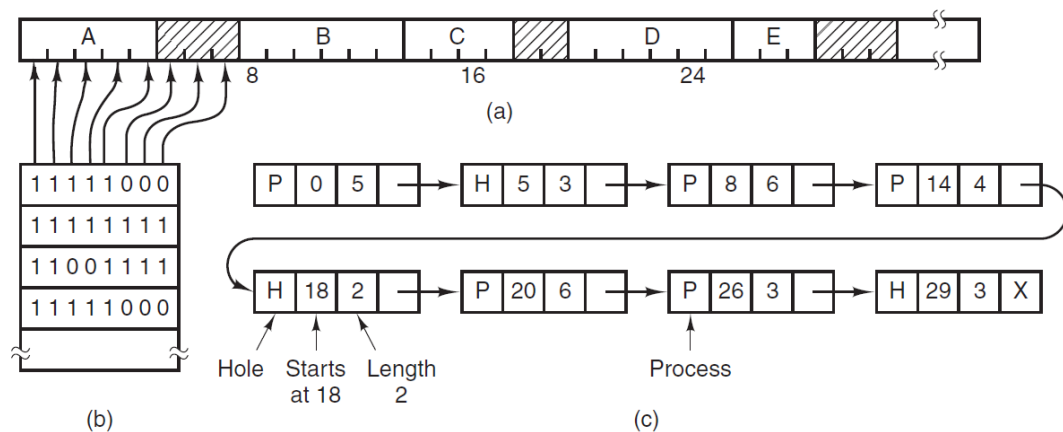


Figure 3-6. (a) A part of memory with five processes and three holes. The tick marks show the memory allocation units. The shaded regions (0 in the bitmap) are free. (b) The corresponding bitmap. (c) The same information as a list.

The figure shows the same memory state in three forms:

Part	Meaning
(a) Memory	Processes <i>A</i> to <i>E</i> occupy some regions; shaded regions are holes.
(b) Bitmap	Each allocation unit has one bit: free or occupied.
(c) Linked list	Each segment is stored as a process entry <i>P</i> or hole entry <i>H</i> .

What 8, 16, and 24 Mean

The numbers under the memory bar are **allocation-unit addresses**. They mark positions in memory, not process names or byte values.

For example, the label 8 means “allocation unit number 8.” Process *B* starts at unit 8, and process *C* starts at unit 14. The labels 8, 16, and 24 are shown as reference marks to make counting easier, like marks on a ruler.

Reading the linked-list entries

Each linked-list node in Fig. 3-6(c) has this form:

Type	Start	Length	Next
------	-------	--------	------

Field	Meaning
Type	<i>P</i> means process; <i>H</i> means hole.
Start	The allocation-unit number where the segment begins.
Length	Number of allocation units in the segment.
Next	Pointer to the next list entry; <i>X</i> means end of list.

Entry	How to read it
<i>P</i> 0 5	Process <i>A</i> starts at unit 0 and occupies units 0, 1, 2, 3, 4. So this is a process segment with start 0 and length 5.
<i>H</i> 5 3	The shaded hole after <i>A</i> starts at unit 5 and covers units 5, 6, 7. So this is a hole with start 5 and length 3.
<i>P</i> 8 6	Process <i>B</i> starts at unit 8 and occupies units 8 through 13. So this is a process segment with start 8 and length 6.
<i>H</i> 18 2	The hole after <i>C</i> starts at unit 18 and covers units 18, 19. So this is a hole with start 18 and length 2.

How to Count Length

The length is counted in allocation units, not bytes. For example, a segment covering units 18 and 19 has length 2, so its list entry is *H* 18 2.

3 Method 1: Bitmaps

Bitmap

A **bitmap** divides memory into fixed-size allocation units. Each unit has one bit in the map.

Bit value	Meaning
0	Unit is free.
1	Unit is occupied.

Some systems may reverse the meaning of 0 and 1. The important idea is that one bit describes one allocation unit.

4 Allocation Unit Size

The allocation unit size is a design choice.

Choice	Effect
Small allocation unit	Less internal waste, but a larger bitmap.
Large allocation unit	Smaller bitmap, but more wasted space in the last unit of a process.

Bitmap Size Idea

one allocation unit $\longleftrightarrow$ one bitmap bit

For example, if the allocation unit is 4 bytes, then 32 bits of memory need only 1 bitmap bit. So the bitmap uses about $1/32$ of memory.

5 Bitmap Strength and Weakness

Point	Explanation
Strength	Bitmap size is fixed once memory size and allocation-unit size are known.
Strength	Simple representation.
Weakness	To allocate a k -unit process, the OS must find k consecutive free bits.
Weakness	Searching for a long run of 0 bits can be slow.

Main Bitmap Cost

Finding a long consecutive free run is the expensive operation.

6 Method 2: Linked Lists

Linked-List Representation

A linked list stores memory as a sequence of segments. Each segment is either a process P or a hole H .

Each list entry stores:

- type: P for process or H for hole;
- starting address;
- length;
- pointer to the next segment.

Sorted by Address

Keeping the list sorted by address makes merging adjacent holes easier when a process exits or is swapped out.

7 When a Process Terminates

When process X terminates, its memory becomes free. The OS may need to merge neighboring holes.

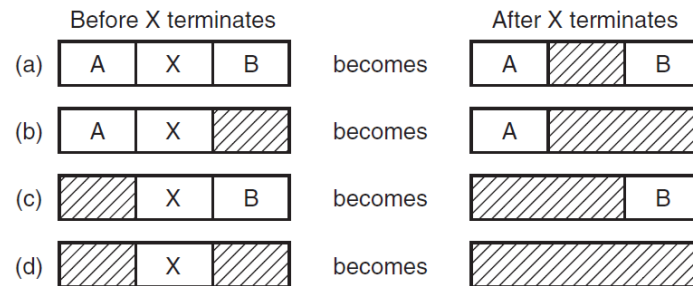


Figure 3-7. Four neighbor combinations for the terminating process, X .

Case	List update
(a) Process on both sides	Replace X 's process entry with a hole entry.
(b) Hole on the right	Merge X 's entry with the right hole.
(c) Hole on the left	Merge the left hole with X 's entry.
(d) Holes on both sides	Merge left hole, X , and right hole into one large hole.

Coalescing

Coalescing means combining adjacent free holes into one larger hole.

8 Single vs Double Linked List

A process table entry often points to the list entry for that process. When the process exits, the OS must check neighboring entries.

List type	Effect
Single linked list	Easy to find the next segment, harder to find the previous segment.
Double linked list	Easier to check both neighbors and merge holes.

9 Allocation Algorithms

When a process is created or swapped in, the memory manager must choose a hole large enough for it.

9.1 First Fit

First Fit

First fit scans from the start of the list and uses the first hole that is large enough.

If the hole is larger than needed, it is split into:

- the allocated process segment;
- the remaining smaller hole.

Why It Is Fast

First fit searches as little as possible.

9.2 Next Fit

Next Fit

Next fit is like first fit, but the next search starts where the previous search ended.

It avoids always starting from the beginning, but simulations show it can perform slightly worse than first fit.

9.3 Best Fit

Best Fit

Best fit searches the entire list and chooses the smallest hole that is large enough.

Best fit tries to avoid wasting a large hole on a small request.

Surprising Problem

Best fit often creates many tiny holes that are too small to be useful.

9.4 Worst Fit

Worst Fit

Worst fit chooses the largest available hole.

The goal is to leave a remaining hole that is still large enough to be useful. In practice, simulations show that worst fit is usually not a good strategy.

10 Example: First Fit vs Best Fit

Using the memory in Fig. 3-6, suppose a block of size 2 is requested.

Algorithm	Choice
First fit	Chooses the first large enough hole, starting at 5.
Best fit	Chooses the smallest sufficient hole, starting at 18.

11 Separate Lists for Processes and Holes

Allocation can be faster if the OS keeps separate lists:

- one list for processes;
- one list for holes.

Benefit	Cost
Allocation scans only holes.	Deallocation becomes more complex.
Best fit can be faster if holes are sorted by size.	Finding neighbors for merging becomes harder.

If the hole list is sorted by size, best fit becomes fast: the first hole that fits is already the smallest adequate hole.

12 Storing List Data Inside Holes

When holes have their own list, the OS can store hole metadata inside the hole itself.

Hole word	Stores
First word	Hole size.
Second word	Pointer to the next hole.

This avoids separate list nodes for holes.

13 Quick Fit

Quick Fit

Quick fit keeps separate lists for common hole sizes, such as 4 KB, 8 KB, 12 KB, and so on.

Finding a hole of a common size becomes very fast.

Quick Fit Problem

When a process terminates, finding neighboring holes for merging can be expensive. Without merging, memory quickly fragments into many small holes.

14 Summary Table

Method	Main idea	Main issue
Bitmap	One bit per allocation unit.	Searching for long free runs is slow.
Linked list	Track process and hole segments.	Must update and merge entries.
First fit	Use first adequate hole.	May leave scattered holes.
Best fit	Use smallest adequate hole.	Creates many tiny holes.
Worst fit	Use largest hole.	Usually poor performance.
Quick fit	Lists for common sizes.	Fast allocation, harder merging.

Virtual Memory

1 Core Idea

Definition

Virtual memory is a memory abstraction that lets each process have its own address space, divided into fixed-size chunks called **pages**.

Not every page must be in physical memory at the same time. The operating system keeps needed pages in RAM and stores the rest on disk.

Big Idea

program can run even if only part of it is in RAM

2 Why Virtual Memory Is Needed

Base and limit registers give each process an address space, but they do not solve the problem of programs and process collections being larger than RAM.

Problem	Explanation
Large programs	A program may need more memory than physical RAM can hold.
Many processes	Each process may fit alone, but all processes together may exceed RAM.
Slow swapping	Swapping whole multi-GB processes to disk can take seconds.

Why Swapping Is Not Enough

Swapping moves whole processes. Virtual memory moves smaller pieces, so the system avoids copying an entire large process every time.

3 Before Virtual Memory: Overlays

Overlay

An **overlay** is a manually separated piece of a large program that is loaded only when needed.

With overlays, the program starts by loading an overlay manager. The manager loads overlay 0, then later loads overlay 1, overlay 2, and so on.

Step	What happens
1	Load the overlay manager.
2	Load and run the first required overlay.
3	Replace or add overlays as execution continues.
4	Keep unused overlays on disk.

Overlay Problem

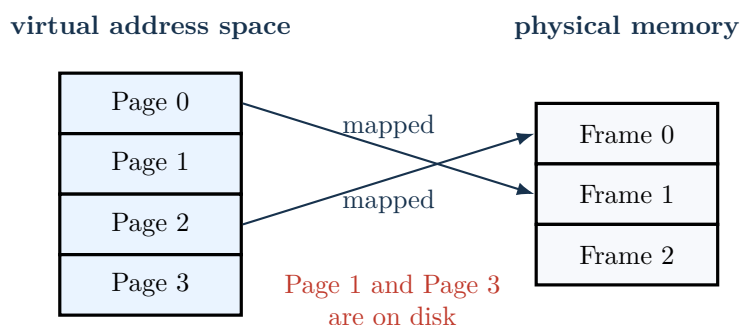
The programmer had to split the program manually. This was time consuming, difficult, and error prone.

4 Pages

Page

A **page** is a contiguous range of addresses inside a process's virtual address space.

Virtual memory splits each process address space into pages. Physical memory is divided into matching slots called page frames.



Key Point

A page can be in RAM or on disk. The process still uses the same virtual addresses either way.

5 How Virtual Memory Runs a Program

When a process references memory, the hardware checks whether the referenced page is already in RAM.

Case	Action
Page is in RAM	Hardware translates the virtual address to a physical address and continues.
Page is not in RAM	A page fault occurs; the operating system loads the missing page from disk.

6 Page Fault

Page Fault

A **page fault** occurs when a process references a virtual page that is not currently in physical memory.

Not every page fault means the same thing. The operating system must check whether the referenced virtual address is valid for the process.

Situation	OS response
Valid page, not in RAM	Load the page from disk into RAM, then restart the instruction.
Invalid address	Raise an error such as a segmentation fault or access violation.

The failed instruction is not simply skipped. After the missing page is loaded, the instruction is normally restarted.



Result

The program behaves as if the page had been in memory all along, except that the page fault causes a delay.

7 Virtual Memory vs Swapping

Method	Unit moved	Main behavior
Swapping	Whole process	A complete process is moved between RAM and disk.
Virtual memory	Page	Only needed pieces of a process are moved.

Main Advantage

Virtual memory is more flexible because RAM can contain pieces of many different processes at the same time.

8 Relation to Base and Limit

Virtual memory generalizes the base-and-limit idea.

Base and limit	Virtual memory
One simple relocation region per process.	Many small pages can be mapped independently.
Whole process usually needs a contiguous memory region.	Pages can be placed in different page frames.
All needed memory should be in RAM.	Only currently needed pages must be in RAM.

9 Multiprogramming Benefit

Virtual memory works well with multiprogramming. RAM can hold pages from many processes at once.

- Process P_1 may have some pages in RAM and others on disk.
- Process P_2 may also have only its active pages in RAM.
- If P_1 waits for a page to be loaded, the CPU can run P_2 .

CPU Utilization

While one process waits for disk I/O, another process can use the CPU.

10 Summary

Concept	Meaning
Virtual memory	Address-space abstraction where only some pages need to be in RAM.
Page	Fixed-size chunk of a virtual address space.
Page frame	Physical-memory slot that can hold one page.
Page fault	Trap caused by referencing a page not currently in RAM; invalid addresses cause an access error.
Main benefit	Programs larger than RAM can run, and many processes can share RAM efficiently.

Paging

1 Core Idea

Definition

Paging is a virtual-memory technique that divides a process's virtual address space into fixed-size **pages** and physical memory into same-size **page frames**.

Programs generate **virtual addresses**. With paging, these virtual addresses are not sent directly to memory. They first go through the **Memory Management Unit (MMU)**, which translates them into physical addresses.

Basic Mapping

$$\text{virtual address} \xrightarrow{\text{MMU}} \text{physical address}$$

2 MMU

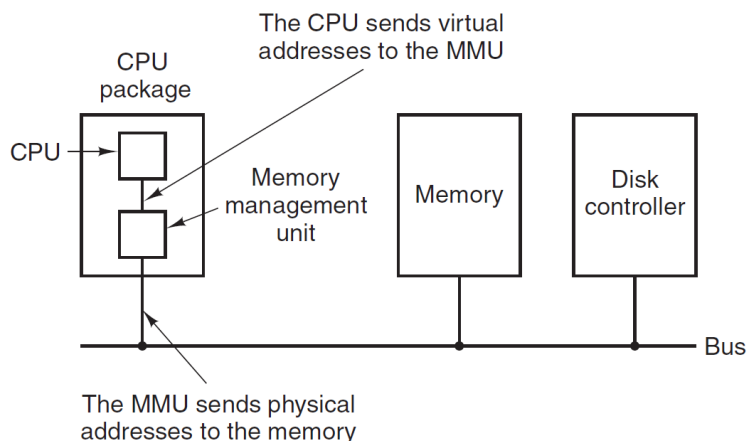


Figure 3-8. The position and function of the MMU. Here the MMU is shown as being a part of the CPU chip because it commonly is nowadays. However, logically it could be a separate chip and was years ago.

What the MMU Does

The CPU sends virtual addresses to the MMU. The MMU translates them and sends physical addresses to memory.

Component	Role
CPU	Generates virtual addresses while executing instructions.
MMU	Maps virtual addresses to physical addresses.
Memory	Sees only physical addresses. It does not know about virtual addresses.
Disk	Stores pages that are not currently in RAM.

3 Virtual Pages and Page Frames

Key Terms

Virtual page: a fixed-size block of virtual addresses.

Page frame: a fixed-size block of physical memory that can hold one page.

Pages and page frames normally have the same size. In the example below:

- virtual address space = 64 KB;
- physical memory = 32 KB;
- page size = 4 KB;
- number of virtual pages = $64/4 = 16$;
- number of physical frames = $32/4 = 8$.

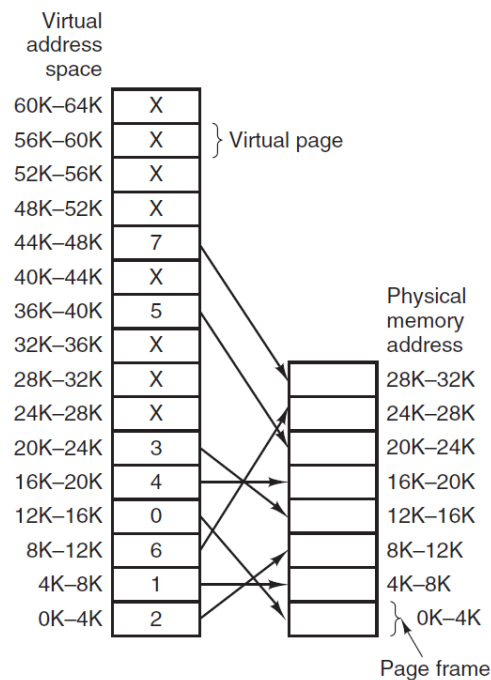


Figure 3-9. The relation between virtual addresses and physical memory addresses is given by the **page table**. Every page begins on a multiple of 4096 and ends 4095 addresses higher, so 4K-8K really means 4096-8191 and 8K to 12K means 8192-12287.

4 Reading Page Ranges

The notation $0K - 4K$ means virtual or physical addresses 0 through 4095. It does not include address 4096.

Range	Actual addresses
0K – 4K	0 to 4095
4K – 8K	4096 to 8191
8K – 12K	8192 to 12287

Why Powers of Two Help

With 4-KB pages, each page has $4096 = 2^{12}$ addresses. This makes the lower 12 bits of an address the offset inside the page.

5 Page Table

Page Table

A **page table** records where each virtual page is located in physical memory, or whether it is absent.

In Fig. 3-9, each virtual page entry contains either:

- a page-frame number, if the page is present in RAM;
- X , if the page is not currently mapped to RAM.

Present/Absent Bit

Real hardware uses a present/absent bit. If the bit says “absent,” a reference to that page causes a page fault.

6 Address Translation Formula

Let:

- V = virtual address;
- P = virtual page number;
- O = offset inside the page;
- F = physical page-frame number;
- page size = 4096 bytes.

Paging Translation

$$\begin{aligned}
 P &= \lfloor V/4096 \rfloor \\
 O &= V \bmod 4096 \\
 \text{physical address} &= F \times 4096 + O
 \end{aligned}$$

7 Example 1: Virtual Address 0

Problem

Translate virtual address 0. From Fig. 3-9, virtual page 0 maps to page frame 2.

Solution

$$P = \lfloor 0/4096 \rfloor = 0$$

$$O = 0 \bmod 4096 = 0$$

Virtual page 0 maps to frame 2, so $F = 2$.

$$\text{physical address} = 2 \times 4096 + 0 = 8192$$

Answer

MOV REG,0 uses physical address 8192.

8 Example 2: Virtual Address 8192

Problem

Translate virtual address 8192. From Fig. 3-9, virtual page 2 maps to page frame 6.

Solution

$$P = \lfloor 8192/4096 \rfloor = 2$$

$$O = 8192 \bmod 4096 = 0$$

Virtual page 2 maps to frame 6, so $F = 6$.

$$\text{physical address} = 6 \times 4096 + 0 = 24576$$

Answer

MOV REG,8192 is effectively transformed into MOV REG,24576.

9 Example 3: Virtual Address 20500

Problem

Translate virtual address 20500. This address is in virtual page 5, and virtual page 5 maps to page frame 3.

Solution

$$P = \lfloor 20500/4096 \rfloor = 5$$

$$O = 20500 - (5 \times 4096)$$

$$O = 20500 - 20480 = 20$$

Virtual page 5 maps to frame 3, so $F = 3$.

$$\text{physical address} = 3 \times 4096 + 20 = 12288 + 20 = 12308$$

Answer

Virtual address 20500 maps to physical address 12308.

10 Unmapped Pages and Page Faults

Only 8 physical frames exist in this example, so only 8 of the 16 virtual pages can be in RAM at once. Entries marked X are absent.

Page Fault

A **page fault** occurs when the MMU sees that the referenced virtual page is not currently mapped to a physical frame.

Example:

```
MOV REG, 32780
```

Virtual address 32780 is 12 bytes into virtual page 8:

$$32780 = 32768 + 12$$

But virtual page 8 is marked X , so the MMU traps to the operating system.

11 Handling a Page Fault

When a valid page is absent, the operating system handles the fault.

Step	Action
1	Choose a page frame to reuse, often one that has not been used recently.
2	If the old page in that frame was modified, write it back to disk.
3	Read the needed page from disk into the freed frame.
4	Update the page table entry.
5	Restart the instruction that caused the page fault.

Example Result

If virtual page 8 is loaded into physical frame 1, then virtual address 32780 maps to $4096 + 12 = 4108$.

12 Inside the MMU

For a 16-bit virtual address with 4-KB pages:

- page size = $4096 = 2^{12}$, so the offset uses 12 bits;
- there are 16 virtual pages, so the page number uses 4 bits;
- physical memory is 32 KB = 8 frames, so the frame number uses 3 bits.

Address part	Meaning
High 4 bits	Virtual page number.
Low 12 bits	Offset inside the page.
Page table result	Physical frame number.
Physical address	Frame number plus unchanged offset.

Key Detail

The offset is copied unchanged. Only the page number is translated into a frame number.

13 Summary

Concept	Meaning
Paging	Divide virtual memory into pages and physical memory into frames.
MMU	Hardware that translates virtual addresses to physical addresses.
Page table	Data structure that maps virtual pages to frames.
Offset	Position inside a page; copied unchanged during translation.
Present/absent bit	Says whether the page is currently in RAM.
Page fault	Trap caused by referencing a valid page that is absent from RAM.

Page Tables

1 Core Idea

Definition

A **page table** maps virtual page numbers to physical page frame numbers.

The MMU uses the page table during address translation. The virtual address is split into:

- a **virtual page number**: high-order bits;
- an **offset**: low-order bits.

Page Table as a Function

$$\text{page table}(\text{virtual page number}) = \text{page frame number}$$

2 Address Split

With a 16-bit virtual address and a 4-KB page size:

Address part	Meaning
Upper 4 bits	Select one of 16 virtual pages.
Lower 12 bits	Select byte offset 0 to 4095 inside that page.

Why 12 Offset Bits?

4 KB = 4096 = 2^{12} , so 12 bits are needed to identify any byte inside one page.

Different splits are possible. More page-number bits means more pages. More offset bits means larger pages.

3 Translation Steps

Step	Action
1	Split the virtual address into page number and offset.
2	Use the virtual page number as an index into the page table.
3	Read the page table entry.
4	If the page is present, get the page frame number.
5	Combine page frame number with the unchanged offset.

Address Construction

$$\text{physical address} = \text{page frame number} \parallel \text{offset}$$

Here $\parallel$ means concatenation: the page frame number becomes the high-order part of the physical address, and the offset remains the low-order part.

4 Inside the MMU

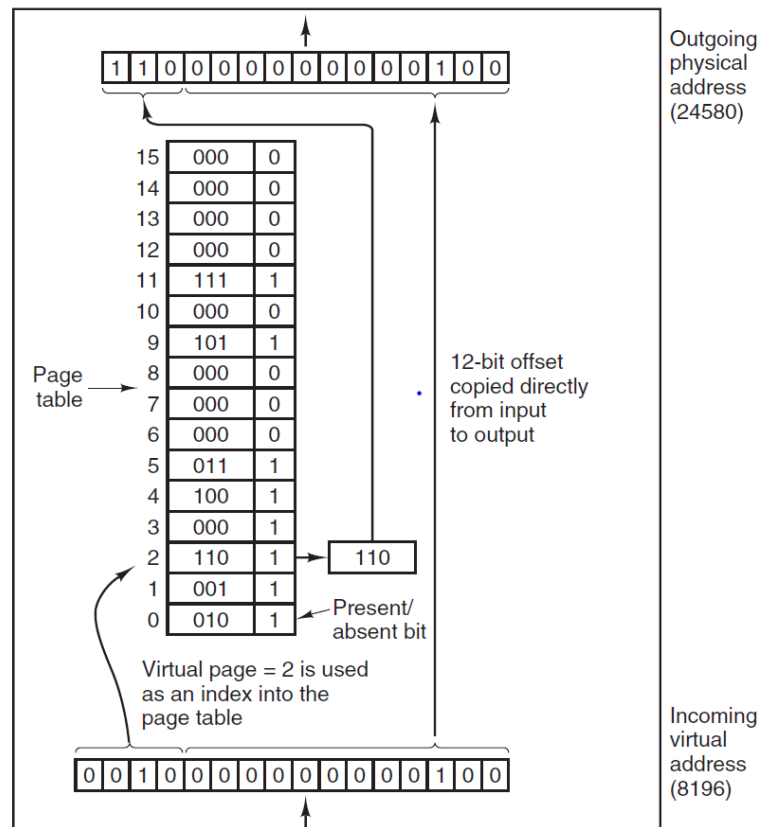


Figure 3-10. The internal operation of the MMU with 16 4-KB pages.

In the figure, the incoming virtual address is 8196. In binary it is:

0010 0000 0000 0100

The upper 4 bits are:

$$0010_2 = 2$$

So virtual page 2 is used as the page-table index. The page table entry for virtual page 2 contains frame number:

$$110_2 = 6$$

The lower 12 bits are the offset:

$$0000\ 0000\ 0100_2 = 4$$

Therefore:

$$\text{physical address} = 6 \times 4096 + 4 = 24580$$

Key Detail

The offset is copied directly from the virtual address to the physical address. Only the page number is replaced.

5 When the Page Is Absent

The page table entry also says whether the page is currently in RAM.

Present/absent bit	Meaning
1	Entry is valid; translation can continue.
0	Page is not in memory; page fault occurs.

Page Fault

If the present/absent bit is 0, the MMU traps to the operating system instead of producing a physical address.

6 Page Table Entry

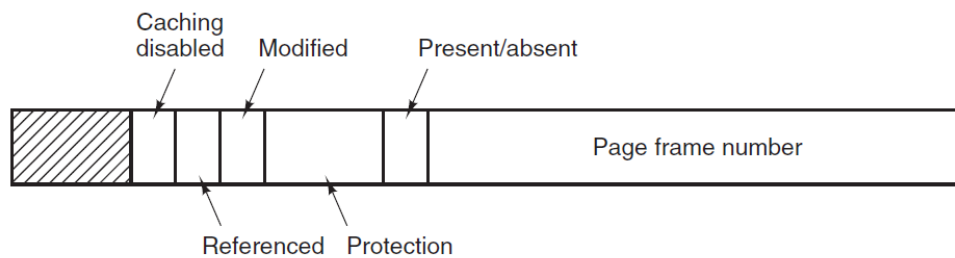


Figure 3-11. A typical page table entry.

Page Table Entry

A **page table entry (PTE)** stores the information needed to translate and control access to one virtual page.

The exact layout is machine dependent, but common fields appear on many systems.

7 Important Fields

Field	Purpose
Page frame number	Tells which physical frame holds the page. This is the main output of translation.
Present/absent bit	Says whether the page is currently in physical memory.
Protection bits	Control allowed access, such as read, write, and execute.
Modified bit	Set when the page is written; also called the dirty bit.
Referenced bit	Set when the page is read or written. Helps choose pages for replacement.
Caching disabled bit	Turns off caching for special pages, such as memory-mapped device registers.

8 Protection Bits

Protection bits define which operations are legal.

Protection style	Meaning
Simple form	One bit: read/write or read-only.
R/W/X form	Separate bits allow reading, writing, and executing.

Access Violation

If a process tries an operation not allowed by the protection bits, the hardware traps to the operating system.

9 Modified and Referenced Bits

Modified bit

The **Modified bit** is set when a page is written.

State	Replacement action
Modified/dirty	Must be written back to disk before eviction.
Clean	Can be discarded because the disk copy is already valid.

Referenced bit

The **Referenced bit** is set when a page is read or written. It helps the OS estimate whether a page is actively being used.

Replacement Use

When choosing a page to evict, the OS prefers pages that have not been referenced recently.

10 Caching Disabled Bit

Some virtual pages map to device registers instead of ordinary memory. In that case, caching can be dangerous.

Reason

If a device register is cached, the CPU may repeatedly read an old cached value instead of the current device value.

The caching disabled bit forces the hardware to fetch the value from the device or memory location every time.

11 What Is Not in the Page Table

The disk address of an absent page is usually not stored in the hardware page table.

Information	Stored where?
Hardware translation data	Page table entry.
Disk location of absent page	Operating-system software tables.

Reason

The page table stores what the hardware needs for address translation. Page-fault recovery information is managed by the operating system.

12 Summary

Concept	Meaning
Page table	Maps virtual page numbers to physical frame numbers.
Virtual page number	Index into the page table.
Offset	Copied unchanged into the physical address.
Present/absent bit	Determines whether translation succeeds or causes a page fault.
Modified bit	Tells whether the page must be written back before eviction.
Referenced bit	Helps page replacement algorithms identify active pages.
Protection bits	Control read, write, and execute permissions.

Speeding Up Paging

1 Core Problem

Main Issue

Paging is powerful, but it creates two implementation problems:

- address translation must be very fast;
- page tables can become very large.

Every instruction must be fetched from memory, and many instructions also read or write data in memory. Therefore virtual-address translation happens extremely often.

Why Speed Matters

If every memory access first requires a slow page-table lookup, paging can become the main performance bottleneck.

2 Why Page Tables Are Large

Modern machines use large virtual addresses. With a fixed page size, more address bits mean more virtual pages.

Address space	Effect on page table
32-bit virtual address	With 4-KB pages, there are about 2^{20} pages, or about one million page-table entries.
64-bit virtual address	The possible virtual address space is much larger, so a simple one-entry-per-page table becomes enormous.
Per process	Each process has its own address space, so each process needs its own page-table information.

Number of Virtual Pages

If the virtual address space has size S bytes and the page size is P bytes, then

$$\text{number of virtual pages} = \frac{S}{P}$$

3 Two Simple Designs

3.1 Page Table in Hardware Registers

Design

Store the whole page table in fast hardware registers, one register entry per virtual page.

Point	Explanation
Advantage	Translation is fast because the MMU does not need extra memory references to read page-table entries.
Problem 1	Large page tables require too many hardware registers, making the design expensive and impractical.
Problem 2	On every context switch, the operating system would have to load the full page table into registers.

3.2 Page Table in Main Memory

Design

Keep the page table in RAM. The CPU/MMU has a register pointing to the start of the current process's page table.

Point	Explanation
Advantage	Context switching is easier: the OS changes one page-table pointer register.
Problem	Each memory reference may require another memory reference to read the page-table entry.
Result	Simple and flexible, but too slow without a cache for translations.

The Basic Tradeoff

Hardware page tables are fast but too expensive for large address spaces. Memory-resident page tables are flexible but need help to avoid slow translation.

4 Translation Lookaside Buffer

Definition

A **Translation Lookaside Buffer (TLB)** is a small, fast hardware cache inside or near the MMU that stores recently used virtual-page to page-frame translations.

The TLB works because programs usually show **locality**: they repeatedly use a small set of pages for current code, data, and stack.

Key Intuition

Most memory references are not spread evenly over the whole page table. A small number of page-table entries are used again and again.

5 Typical TLB Entry

A TLB entry stores the information needed to translate and protect one virtual page.

Field	Meaning
Valid bit	Says whether this TLB slot currently contains a usable entry.
Virtual page number	The virtual page being cached. This is needed so the MMU can compare the requested page with the TLB entry.
Modified bit	Records whether the page has been written to. It may later be copied back to the page table.
Protection bits	Record allowed operations, such as read, write, and execute.
Page frame number	The physical frame where the page currently resides.

5.1 Example TLB Contents

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	RX	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	RX	50
1	21	0	RX	45
1	860	1	RW	14
1	861	1	RW	75

This kind of TLB might occur when a process is repeatedly executing code on pages 19, 20, and 21, using data on pages 129, 130, and 140, and using stack pages 860 and 861.

6 TLB Lookup

TLB First Rule

virtual page $\rightarrow$ check TLB first $\rightarrow$ $\begin{cases} \text{hit: use cached frame} \\ \text{miss: consult page table} \end{cases}$

When the CPU produces a virtual address, the MMU extracts the virtual page number and searches the TLB.

Case	What happens
TLB hit	The virtual page is found in the TLB. If protection allows the access, the MMU immediately uses the cached page frame number.
Protection fault	The page is in the TLB, but the requested operation is not allowed, such as writing to a read-only page. The CPU traps to the OS.
TLB miss	The virtual page is not in the TLB. The MMU or OS must look in the page table.

7 What Happens on a TLB Miss

7.1 If the Page Is Present in RAM

The page table contains a valid mapping, but that mapping was not cached in the TLB.

1. Look up the virtual page in the page table.
2. Choose a TLB entry to replace.
3. Copy the page-table entry into the TLB.
4. Restart or continue the memory reference.

Result

No disk access is needed. The page is already in RAM; only the TLB lacked the translation.

7.2 If the Page Is Not Present in RAM

The page table entry says the page is absent. This is a page fault.

1. Trap to the operating system.
2. Find or free a physical page frame.
3. Bring the missing page from disk into RAM.
4. Update the page table.
5. Update or refill the TLB entry.
6. Restart the faulting instruction.

Important Difference

A TLB miss is not automatically a page fault. A TLB miss means the translation is not in the TLB. A page fault means the page is not currently mapped in RAM, or the access is invalid.

8 Hardware TLB Management

Hardware-Managed TLB

In a hardware-managed TLB system, the MMU handles ordinary TLB misses by walking the page table and loading the needed TLB entry.

The operating system is normally involved only when the page is absent from RAM or when a protection violation occurs.

Event	Handler
TLB hit	MMU hardware handles it directly.
TLB miss, page present	MMU hardware performs a page-table walk and fills the TLB.
Page absent	OS handles the page fault.
Invalid or disallowed access	OS handles the fault, often by terminating the process.

9 Software TLB Management

Software-Managed TLB

In a software-managed TLB system, a TLB miss traps to the operating system. The OS finds the mapping, chooses a TLB slot, inserts the new entry, and restarts the instruction.

This approach was used on several RISC architectures. It simplifies the MMU because page-table searching is moved into operating-system software.

Advantage	Cost
Simpler MMU	TLB misses trap into the OS, so the miss handler must be very fast.
More OS control	Frequent misses can hurt performance.
More chip area for caches	The OS must carefully avoid extra TLB misses while handling a TLB miss.

10 Soft Misses and Hard Misses

Miss type	Meaning	Cost
Soft miss	Page is in RAM, but the translation is not in the TLB.	Fast: update the TLB; no disk I/O.
Hard miss	Page is not in RAM.	Slow: the OS must read the page from disk.
Minor page fault	Page is in RAM somewhere, but not mapped in this process's page table yet.	Slower than a soft TLB miss, but no disk read is needed.
Major page fault	Page must be fetched from disk.	Very slow compared with CPU instructions.
Invalid access	The process used an address it is not allowed to access.	OS raises an error such as segmentation fault or access violation.

Common Confusion

A hard miss is much more expensive than a soft miss because it needs disk I/O. A soft miss only repairs missing translation information.

11 Page Table Walk

Definition

A **page table walk** is the lookup procedure used to find the page-table entry for a virtual page.

With hardware TLB management, the MMU performs the walk. With software TLB management, the operating system may perform the walk after a TLB fault.

Why Walks Matter

If page-table pages themselves are not already easy to access, walking the page table can trigger additional translation work. This is one reason systems use caches and careful TLB miss handlers.

12 Reducing TLB Miss Cost

Systems may use several practical tricks:

Technique	Idea
Use a moderately large TLB	More entries reduce the chance that a useful translation has been evicted.
Preload likely entries	The OS may load entries for code, data, or stack pages that are likely to be used soon.
Software TLB cache	The OS can keep a software cache of mappings in a fixed location that is easy for the miss handler to access.
Exploit locality	Programs tend to reuse current code, data, and stack pages, so recent translations are valuable.

13 Big Picture

Mechanism	Main purpose	Main caveat
Page table	Complete mapping from virtual pages to page frames.	Large and too slow to consult on every access.
TLB	Small fast cache of recent translations.	Misses still need page-table lookup or OS handling.
Hardware TLB handling	Fast common-case miss handling by MMU.	More complex MMU hardware.
Software TLB handling	Simpler MMU and more OS control.	Miss handler must be extremely efficient.

Final Summary

Paging needs fast translation and compact page-table management.
The TLB speeds paging by caching recently used translations.
TLB miss $\neq$ page fault: the missing translation may still point to a page in RAM.

Page Tables for Large Memories

1 Core Problem

Problem

TLBs make address translation faster, but they do not solve the size problem: large virtual address spaces can require huge page tables.

If every virtual page needs one page-table entry, a large virtual address space quickly becomes expensive.

Issue	Meaning
Fast translation	The MMU must translate every memory reference quickly.
Large page tables	A large virtual address space has many virtual pages.
Per-process tables	Each process has its own address space, so page-table storage is repeated per process.

Page Count

$$\text{number of virtual pages} = \frac{\text{virtual address space size}}{\text{page size}}$$

2 Two Main Solutions

Method	Main idea	Best when
Multilevel page table	Split the page table into levels and allocate lower-level tables only for used regions.	The virtual address space is sparse.
Inverted page table	Keep one entry per physical page frame instead of one entry per virtual page.	Physical memory is much smaller than the virtual address space.

3 Multilevel Page Tables

Definition

A **multilevel page table** breaks one large page table into a hierarchy. The top level points to lower-level page tables only where the process actually uses memory.

The goal is simple: do not keep page tables for huge unused regions of the virtual address space.

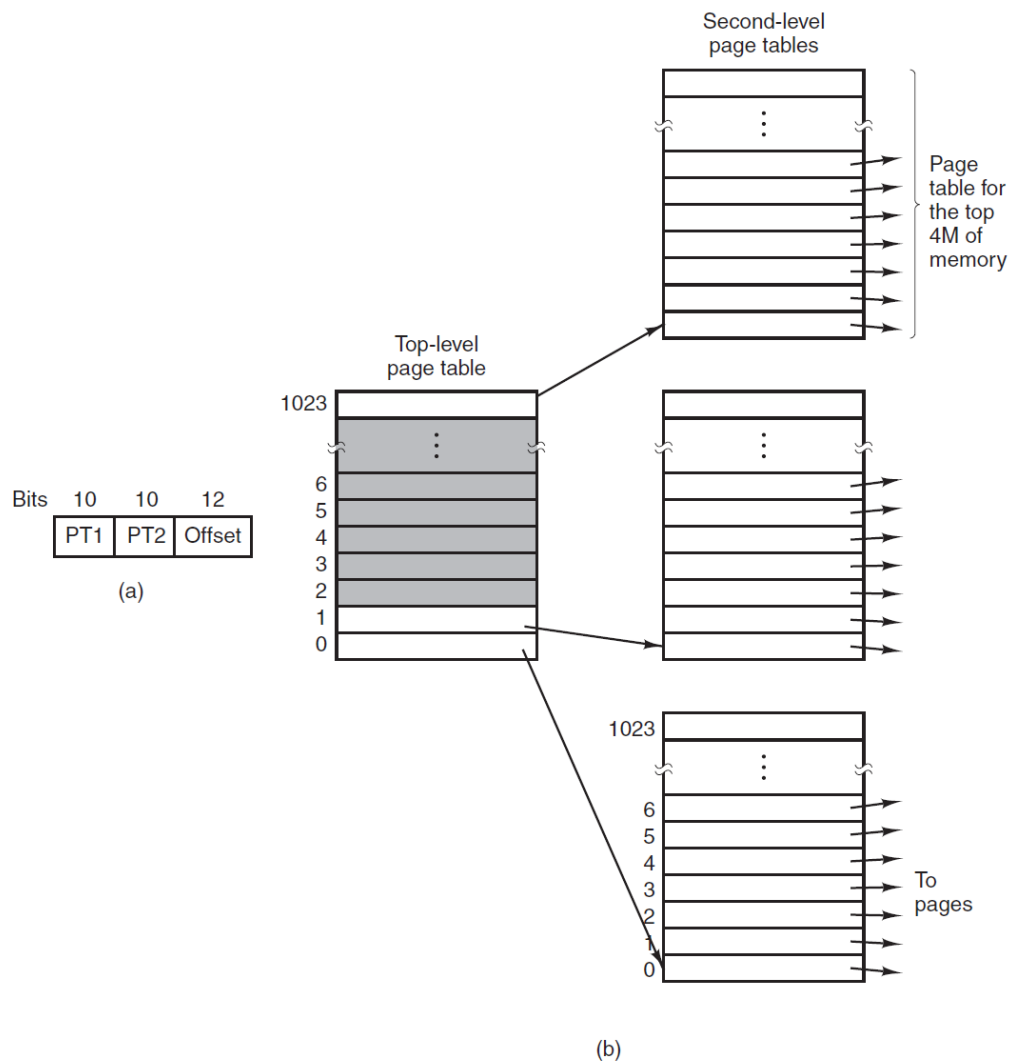


Figure 3-13. (a) A 32-bit address with two page table fields. (b) Two-level page tables.

4 Address Split

In the example, a 32-bit virtual address is split into three fields:

Field	Bits	Meaning
PT1	10 bits	Index into the top-level page table.
PT2	10 bits	Index into the selected second-level page table.
Offset	12 bits	Byte position inside the selected 4-KB page.

Why Pages Are 4 KB

The offset field has 12 bits, so one page contains $2^{12} = 4096$ bytes = 4 KB.

Address Capacity

$$2^{10} \times 2^{10} \times 2^{12} = 2^{32} \text{ addressable bytes}$$

5 How Two-Level Translation Works

1. The MMU extracts the **PT1** field.
2. PT1 indexes the top-level page table.
3. The selected top-level entry points to a second-level page table.
4. The MMU extracts the **PT2** field.
5. PT2 indexes the selected second-level page table.
6. The second-level entry gives the physical page frame number.
7. The page frame number is combined with the unchanged offset.

Final Physical Address

$$\text{physical address} = (\text{page frame number} \times \text{page size}) + \text{offset}$$

6 Why It Saves Space

Suppose a process uses only:

- bottom 4 MB for program text;
- next 4 MB for data;
- top 4 MB for stack.

The middle of the address space is a large unused hole.

Space Saving

Only the top-level page table and the second-level tables for used regions must exist. Unused 4-MB regions do not need second-level page tables.

In the figure, the system needs:

Table	Purpose
Top-level table	Covers the whole 4-GB virtual address space in 4-MB chunks.
Second-level table for entry 0	Maps the program-text region.
Second-level table for entry 1	Maps the data region.
Second-level table for entry 1023	Maps the stack region near the top of memory.

7 Worked Example: Address 0x00403004

Problem

Translate the virtual address 0x00403004 using the two-level split:

$$\text{PT1} = 10 \text{ bits}, \quad \text{PT2} = 10 \text{ bits}, \quad \text{Offset} = 12 \text{ bits}.$$

Solution

First convert the page size:

$$4 \text{ KB} = 4096 = 2^{12}$$

The offset is the lower 12 bits. In hexadecimal, 12 bits correspond to the last 3 hex digits.

$$0x00403004 \Rightarrow \text{offset} = 0x004 = 4$$

Remove the offset by shifting right 12 bits:

$$0x00403004 \div 0x1000 = 0x403$$

Now split the remaining 20 page-number bits into PT1 and PT2:

$$0x403 = 1 \times 1024 + 3$$

So:

$$\text{PT1} = 1, \quad \text{PT2} = 3, \quad \text{Offset} = 4$$

Step	Meaning
PT1 = 1	Select top-level entry 1, which covers virtual addresses 4 MB to 8 MB minus 1.
PT2 = 3	Select page 3 inside that 4-MB chunk.
Offset = 4	Select byte 4 inside that page.

The start of page 3 within the selected 4-MB chunk is:

$$3 \times 4096 = 12288$$

The absolute virtual address is:

$$4 \text{ MB} + 12288 + 4 = 4,194,304 + 12,288 + 4 = 4,206,596$$

Interpretation

The address 0x00403004 means: use top-level entry 1, then second-level entry 3, then byte offset 4 inside that page.

8 Page Faults in Multilevel Tables

Each top-level entry and second-level entry has validity information.

Case	Result
Top-level entry absent	The whole 4-MB region is not mapped. Access causes a fault.
Second-level entry absent	The selected page is not currently present in RAM. Access causes a page fault.
Invalid address	The process is trying to access memory it is not allowed to use. The OS may signal or kill it.

9 More Than Two Levels

Two levels can be extended to three, four, or more levels. More levels make it possible to represent very large sparse address spaces without allocating page-table memory for unused regions.

System idea	Explanation
Two-level paging	A top-level directory points to page tables.
Three-level paging	Adds another directory layer above the page tables.
Four-level paging	Common in 64-bit style address translation, where the address space is much larger.

Tradeoff

More levels save memory for sparse address spaces, but a TLB miss may require more page-table lookups during the page-table walk.

10 Inverted Page Tables

Definition

An **inverted page table** has one entry per physical page frame, not one entry per virtual page.

Instead of asking, “where is virtual page p ?”, the inverted table records, “which process and virtual page currently occupy this physical frame?”

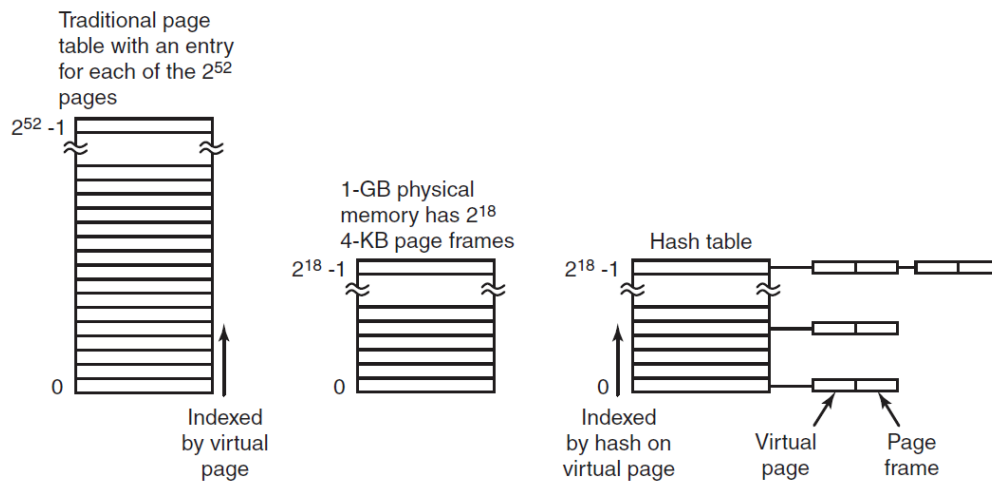


Figure 3-14. Comparison of a traditional page table with an inverted page table.

11 Traditional vs Inverted

Feature	Traditional page table	Inverted page table
Number of entries	One entry per virtual page.	One entry per physical page frame.
Indexed by	Virtual page number.	Usually searched by hashing the process ID and virtual page.
Space cost	Huge for large virtual address spaces.	Depends mainly on physical RAM size.
Lookup difficulty	Direct indexing is easy.	Search is harder without hashing and TLB support.
Typical use	Simple and fast when page-table size is manageable.	Useful for very large virtual address spaces, especially 64-bit systems.

12 Inverted Page Table Example

With:

- 64-bit virtual addresses;
- 4-KB pages;
- 4 GB of RAM.

The number of physical page frames is:

$$4 \text{ GB} = 2^{32} \text{ bytes}$$

$$4 \text{ KB} = 2^{12} \text{ bytes}$$

$$\text{physical frames} = \frac{2^{32}}{2^{12}} = 2^{20} = 1,048,576$$

Meaning

The inverted page table needs about 1,048,576 entries because it has one entry per physical frame, not one entry per possible virtual page.

13 Why Hashing Is Needed

The downside of inverted page tables is lookup.

Lookup Problem

The MMU cannot directly index the table using only the virtual page number, because the table is organized by physical frames.

To find where process n 's virtual page p is located, the system searches for the pair:

$$(n, p)$$

Searching the entire inverted page table on every memory reference would be too slow. A hash table solves this by grouping likely matches into short chains.

Step	Action
1	Hash the process ID and virtual page number.
2	Use the hash value to select a chain of entries.
3	Search only that short chain for the exact (process, virtual page) pair.
4	If found, get the physical page frame number.
5	Insert the translation into the TLB for faster future access.

14 Role of the TLB

Inverted page tables depend heavily on the TLB.

TLB Role

If the requested translation is already in the TLB, the inverted page table is not searched. The expensive search happens mainly on TLB misses.

This is why inverted page tables can be practical: most references hit in the TLB, and only misses need the hash-table lookup.

15 Comparison Summary

Method	Strength	Weakness
Multilevel page table	Saves memory by omitting lower-level tables for unused regions.	Page-table walks may require multiple memory references.
Inverted page table	Table size depends on physical memory, not virtual address space.	Translation is harder and needs TLB plus hashing.

Final Summary

Large virtual address spaces make flat page tables too large.
Multilevel tables save space by allocating only needed lower-level tables.
Inverted tables save space by storing one entry per physical frame.
TLBs are essential because both schemes can make misses expensive.

Page Replacement Algorithms

1 Core Idea

Definition

Page replacement is the operating system's decision about which page to evict from physical memory when a page fault occurs and a free page frame is needed.

When a process references a page that is not currently in RAM, a **page fault** occurs. The operating system must bring the missing page into memory. If no free frame is available, some existing page must be removed.

Basic Fault Handling Idea

page fault → choose victim page → evict victim → load needed page

2 Why Replacement Is Needed

Physical memory is limited. Virtual memory lets processes have large address spaces, but only some pages can be in RAM at one time.

Situation	What the OS must do
Free frame exists	Load the missing page into the free frame.
No free frame exists	Pick an existing page as the victim and evict it.
Victim page is clean	Overwrite it; disk already has a correct copy.
Victim page is dirty	Write it back to disk before reusing the frame.

Main Goal

A good replacement algorithm tries to evict a page that will not be needed again soon.

3 Clean Pages and Dirty Pages

Clean vs Dirty

A **clean page** has not been modified since it was loaded from disk. A **dirty page** has been modified while in memory.

This distinction matters because eviction cost is different.

Page state	Eviction action
Clean	No disk write is needed. The page frame can be reused directly.
Dirty	The page must be written back to disk before the frame is reused.

Why Dirty Pages Are Expensive

Evicting a dirty page requires disk I/O. Disk I/O is far slower than normal memory access, so replacement algorithms often prefer clean victims when possible.

4 Why Random Replacement Is Weak

The OS could choose a random page to evict, but this often performs poorly.

Choice	Consequence
Evict rarely used page	Good choice. The page may not be needed again for a long time.
Evict heavily used page	Bad choice. The page may fault back in almost immediately.

Performance Idea

The cost of a bad eviction is not only the current page fault. It can also create another page fault soon after.

5 What Page Replacement Algorithms Try to Predict

A replacement algorithm does not know the future exactly. It uses available information to estimate which page is least useful.

Information	Why it helps
Recent use	Recently used pages may be used again soon.
Reference bit	Shows whether a page has been accessed recently.
Modified bit	Shows whether eviction requires writing the page back to disk.
Process behavior	Some pages belong to active code, data, or stack regions.

6 Replacement in Other Systems

Page replacement is one example of a broader problem: when a storage layer is full, choose what to remove.

System	What is cached	Replacement question
Virtual memory	Disk pages in RAM	Which page frame should be reused?
CPU cache	Recent memory blocks, often 32 or 64 bytes each	Which cache block should be replaced?
Web server cache	Frequently requested web pages	Which cached web page should be removed?

Same Pattern, Different Speed

The idea is similar across systems, but the time scale differs. CPU cache replacement must happen in nanoseconds, while virtual-memory replacement may involve millisecond-scale disk I/O.

7 Clean and Dirty in Different Caches

Virtual memory has an extra issue: pages in RAM may be modified.

Cache type	Can cached item be modified?	Eviction consequence
Virtual memory	Yes	Dirty pages must be written back to disk.
Web page cache	Usually no	A fresh copy remains on disk or storage, so no write-back is needed.
CPU cache	Yes, depending on cache policy	Dirty cache blocks may need write-back to lower memory.

8 Local vs Global Replacement

When a page must be evicted, the OS has to decide the search scope.

Two Scopes

Local replacement: evict one of the faulting process's own pages.

Global replacement: evict a page from any process in memory.

Policy	Meaning	Main effect
Local replacement	The victim must belong to the process that caused the page fault.	Each process is effectively limited to its own assigned frames.
Global replacement	The victim may belong to another process.	Memory can shift toward processes that currently need more frames.

Important Design Question

Page replacement is not only about which page is old or unused. It is also about whether one process is allowed to take memory frames away from another process.

9 Page Replacement Flow

1. A process references a page.
2. The MMU discovers that the page is not present in RAM.
3. A page fault traps to the operating system.
4. The OS checks for a free frame.
5. If no free frame exists, the OS chooses a victim page.
6. If the victim is dirty, the OS writes it back to disk.
7. The OS loads the needed page into the freed frame.
8. The page table and TLB information are updated.
9. The faulting instruction is restarted.

10 What Makes a Good Victim?

Good victim property	Reason
Not recently used	Less likely to be needed again soon.
Clean	Can be evicted without disk write-back.
Belongs to inactive process	Less likely to be referenced immediately.
Not part of active code/data/stack	Lower chance of immediate refault.

Central Principle

The best victim is a page whose removal causes the least future cost.

11 Final Summary

Concept	Key point
Page fault	The needed page is not currently available in RAM.
Page replacement	The OS chooses a resident page to evict.
Clean page	Can be discarded because disk already has the correct copy.
Dirty page	Must be written back before eviction.
Good algorithm	Avoids evicting pages that will soon be used again.
Local/global choice	Determines whether the victim must come from the faulting process or from any process.

One-Box Summary

Page replacement chooses a victim page when RAM is full.
Clean victims are cheaper than dirty victims.
A good policy avoids evicting heavily used pages.
The OS must also choose local or global replacement scope.

Optimal Page Replacement Algorithm

1 Core Idea

Definition

The **optimal page replacement algorithm** evicts the page whose next use is farthest in the future.

When a page fault occurs, the operating system must choose one page already in memory as the victim. The optimal rule is simple: remove the page that will not be needed for the longest time.

Optimal Rule

evict the resident page with the farthest next reference

2 How the Decision Is Made

Imagine each page in memory has a label showing when it will be used next.

Page	Next use
<i>A</i>	After 10 instructions.
<i>B</i>	After 100 instructions.
<i>C</i>	After 1000 instructions.
<i>D</i>	After 8,000,000 instructions.

Decision

Evict page *D*, because it will be unused for the longest time.

3 Worked Example

Problem

Memory has 3 frames containing *A*, *B*, and *C*. A page fault occurs for *D*. Future references are:

B, *C*, *A*, *B*, *E*, *C*, *A*

Which page should be evicted?

Solution

Resident page	Next use in the future string
<i>A</i>	Third.
<i>B</i>	First.
<i>C</i>	Second.

Page *A* is used farthest in the future, so it is the best victim.

Answer

Evict *A*, then load *D* into the freed frame.

4 Why It Is Impossible in Practice

Main Limitation

The operating system does not know the future page-reference sequence when the page fault happens.

To apply the optimal rule, the OS would need to know exactly:

- which resident page will be used next;
- how long until each resident page is used again;
- whether any resident page will never be used again.

Real systems only know past and current behavior, not exact future references.

5 Why It Is Still Useful

Benchmark

Optimal replacement gives the lowest possible page-fault count for a known reference string.

Researchers can record a program's page references in a simulator, then replay that same reference string and compare real algorithms against the optimal result.

Use

It is not a practical operating-system algorithm. It is a standard for measuring how close practical algorithms are to the best possible result.

6 Final Summary

Point	Meaning
Rule	Evict the page whose next use is farthest away.
Strength	Gives the best possible page-fault performance.
Weakness	Requires future knowledge, so it cannot run in a real OS.
Main use	Benchmark for comparing practical replacement algorithms.

One-Box Summary

Optimal replacement is theoretically best.
It evicts the page needed farthest in the future.
It is useful for comparison, not implementation.

Not Recently Used Page Replacement Algorithm

1 Core Idea

Definition

The **Not Recently Used (NRU)** page replacement algorithm divides pages into classes using the **Referenced** bit R and the **Modified** bit M , then evicts a page from the lowest-numbered nonempty class.

NRU is based on a practical idea: a page that has not been referenced recently is usually a better victim than a page that is currently active.

NRU Rule

choose a random page from the lowest-numbered nonempty class

2 The R and M Bits

Most virtual-memory systems keep two usage bits in each page table entry.

Bit	Name	Meaning
R	Referenced	Set to 1 whenever the page is read or written.
M	Modified	Set to 1 whenever the page is written to.

Hardware Support

The R and M bits must be updated on memory references, so hardware usually sets them automatically.

Once a bit becomes 1, it stays 1 until the operating system clears it.

3 Why the OS Clears R

When a process starts, the OS initializes both bits:

$$R = 0, \quad M = 0$$

Then, periodically, usually on a clock interrupt, the OS clears the R bit.

Purpose of Clearing R

Clearing R lets the OS distinguish pages referenced recently from pages not referenced since the last clock tick.

The M bit is not cleared in the same way because it tells the OS whether the page must be written back to disk before eviction.

4 NRU Classes

When a page fault occurs, NRU inspects pages and sorts them into four classes.

Class	R	M	Meaning
0	0	0	Not referenced recently and not modified. Best victim class.
1	0	1	Not referenced recently but modified.
2	1	0	Referenced recently but not modified.
3	1	1	Referenced recently and modified. Worst victim class.

Class Priority

NRU prefers lower class numbers:

Class 0 $\rightarrow$ Class 1 $\rightarrow$ Class 2 $\rightarrow$ Class 3

5 Why Class 1 Can Exist

At first, class 1 may look impossible:

$$R = 0, \quad M = 1$$

If a page was modified, it must also have been referenced. So how can it be modified but not referenced?

Explanation

A page can first be in class 3, meaning $R = 1, M = 1$. Later, a clock interrupt clears R , but leaves M unchanged. The page becomes class 1: $R = 0, M = 1$.

Key Detail

Clock interrupts clear R , not M . The modified bit must remain available so the OS knows whether disk write-back is needed.

6 Algorithm Steps

1. A page fault occurs.
2. The OS checks the R and M bits of resident pages.
3. Pages are grouped into classes 0, 1, 2, and 3.
4. The OS finds the lowest-numbered class that is not empty.
5. It chooses a random page from that class.

6. If the chosen page is dirty, it is written back to disk.
7. The incoming page is loaded into the freed frame.

7 Worked Example

Problem

Suppose the pages in memory have these bit values:

Page	R	M
A	1	1
B	0	1
C	1	0
D	0	0

Which page class should NRU choose from?

Solution

Page	R	M	Class
A	1	1	Class 3.
B	0	1	Class 1.
C	1	0	Class 2.
D	0	0	Class 0.

The lowest nonempty class is class 0.

Answer

NRU chooses from class 0. In this example, it evicts page D .

8 Why NRU Prefers Class 1 over Class 2

NRU chooses class 1 before class 2, even though class 1 pages are dirty.

Reason

NRU considers “not recently referenced” more important than “clean.” A dirty page not used recently may be a better victim than a clean page that is currently active.

This is a heuristic. It is not always perfect, but it is simple and often adequate.

9 If Hardware Lacks R and M Bits

The OS can simulate these bits using page faults.

Event	OS action
First reference to a page	Cause a page fault, set simulated $R = 1$, and mark the page read-only.
First write to that page	Cause another page fault, set simulated $M = 1$, and mark the page read/write.

Cost

Simulating R and M works, but it is slower than direct hardware support because it uses page faults.

10 Strengths and Weaknesses

Point	Explanation
Strength	Easy to understand and moderately efficient.
Strength	Uses simple hardware-maintained bits.
Weakness	Chooses randomly within a class, so it may not pick the best page.
Weakness	Less accurate than algorithms that track page use more precisely.

11 Final Summary

One-Box Summary

NRU uses R and M bits to classify pages.
Lowest class is preferred: 00, 01, 10, 11.
It evicts randomly from the lowest nonempty class.
It is simple and practical, but not optimal.

First-In, First-Out Page Replacement Algorithm

1 Core Idea

Definition

The **First-In, First-Out (FIFO)** page replacement algorithm evicts the page that has been in memory the longest.

FIFO treats memory like a queue. New pages enter at the tail of the queue. When a page must be evicted, the page at the head of the queue is removed.

FIFO Rule

evict the oldest page in memory

2 Queue Structure

The operating system keeps a list of pages currently in memory.

Position	Meaning
Head	Least recent arrival; this page entered memory earliest.
Tail	Most recent arrival; this page entered memory latest.

Queue Movement

On a page fault, remove from the head and insert the incoming page at the tail.

3 Algorithm Steps

1. A page fault occurs.
2. If a free frame exists, load the needed page into it.
3. If memory is full, choose the page at the head of the FIFO queue.
4. Evict that oldest page.
5. Load the new page.
6. Add the new page to the tail of the queue.

4 Worked Example

Problem

Memory has 3 frames. The FIFO queue is:

Head $[A, B, C]$ Tail

A page fault occurs for page D . Which page is evicted?

Solution

The page at the head is A , so A is the oldest page in memory.

$$[A, B, C] \rightarrow [B, C, D]$$

Answer

FIFO evicts A and inserts D at the tail.

5 Supermarket Analogy

FIFO is like a store with limited shelf space.

Store idea	Memory idea
Shelves can hold only k products.	RAM can hold only k pages.
New product arrives.	New page must be loaded.
Oldest stocked product is removed.	Oldest resident page is evicted.

The analogy shows why FIFO is simple: the system only needs to remember arrival order.

6 Main Weakness

Problem

The oldest page is not necessarily the least useful page.

A page may have entered memory long ago and still be heavily used. FIFO does not check whether the page was referenced recently.

FIFO assumption	Why it can fail
Old pages are good victims.	An old page may still contain important code, data, or stack information.
Arrival time is enough.	FIFO ignores actual page usage after arrival.

Key Point

FIFO has low overhead, but pure FIFO is rarely used because it can evict useful pages.

7 Strengths and Weaknesses

Point	Explanation
Strength	Very easy to implement.
Strength	Low overhead: only a queue of resident pages is needed.
Weakness	Ignores whether a page is actively used.
Weakness	May remove a heavily used page just because it is old.

8 Final Summary

One-Box Summary

FIFO evicts the page that entered memory earliest.
It stores pages in arrival order: head = oldest, tail = newest.
It is simple and cheap, but it ignores actual page usage.

Second-Chance Page Replacement Algorithm

1 Core Idea

Definition

The **second-chance page replacement algorithm** is a modified FIFO algorithm that checks the referenced bit R before evicting the oldest page.

FIFO removes the oldest page even if it is still heavily used. Second chance improves this by giving a recently referenced page one more chance to stay in memory.

Second-Chance Rule

$R = 0$: evict the page
 $R = 1$: clear R and move the page to the tail

2 How It Modifies FIFO

The operating system keeps pages in FIFO order:

head = oldest page, tail = newest page

On a page fault, it examines the page at the head of the list.

State	Action
$R = 0$	The page is old and not recently used, so evict it.
$R = 1$	Clear R , move the page to the tail, update its load time, and continue searching.

Main Intuition

An old page should not be evicted immediately if it was referenced recently.

3 Algorithm Steps

1. A page fault occurs.
2. Look at the oldest page, at the head of the FIFO list.
3. If its R bit is 0, choose it as the victim.
4. If its R bit is 1, clear R .
5. Move that page to the tail of the list, as if it just arrived.
6. Repeat the search with the next oldest page.

4 Example: Page A Gets a Second Chance

Setup

Pages A through H are in memory in FIFO order. The numbers are their load times.

Head $A_0, B_3, C_7, D_8, E_{12}, F_{14}, G_{15}, H_{18}$ Tail

A page fault occurs at time 20.

The oldest page is A , loaded at time 0.

Case	Result
$A.R = 0$	A has not been referenced recently, so evict A .
$A.R = 1$	Clear $A.R$, move A to the tail, and set its new load time to 20.

If $A.R = 1$, the list becomes:

Head $B_3, C_7, D_8, E_{12}, F_{14}, G_{15}, H_{18}, A_{20}$ Tail

Meaning

Page A was old, but it was recently referenced. Second chance delays its eviction and tests page B next.

5 What the Algorithm Is Looking For

Second chance searches for an old page that has not been referenced in the most recent clock interval.

Page type	How second chance treats it
Old and $R = 0$	Good victim. Evict it.
Old but $R = 1$	Recently used. Clear R and move it to the tail.

6 If All Pages Have $R = 1$

Degenerates to FIFO

If every page has $R = 1$, second chance eventually becomes pure FIFO.

Here is why it still terminates:

1. The algorithm starts at the oldest page.
2. Each page has $R = 1$, so each page is moved to the tail and has R cleared.
3. Eventually the algorithm returns to the original oldest page.
4. That page now has $R = 0$, so it is evicted.

Guarantee

Second chance always finds a victim because it clears R bits while scanning.

7 Strengths and Weaknesses

Point	Explanation
Strength	Simple improvement over FIFO.
Strength	Avoids immediately evicting recently used old pages.
Weakness	May scan many pages before finding a victim.
Weakness	If all pages were referenced, it behaves like FIFO.

8 Final Summary

One-Box Summary

Second chance is FIFO plus an R -bit check.
If $R = 0$, evict the oldest page.
If $R = 1$, clear R and move the page to the tail.
It protects recently used pages, but can degrade to FIFO.

Clock Page Replacement Algorithm

1 Core Idea

Definition

The **clock page replacement algorithm** is an efficient implementation of second chance. It keeps page frames in a circular list and uses a moving hand to find a victim.

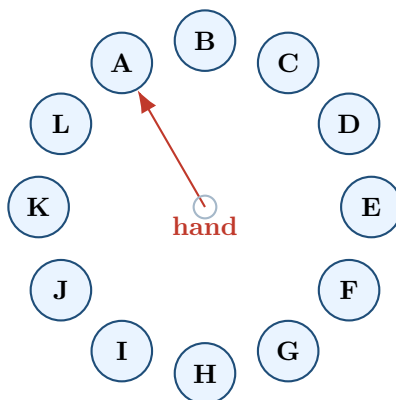
Second chance moves pages around in a list. Clock avoids that overhead by leaving pages in place and moving only a pointer, called the **clock hand**.

Clock Rule

$R = 0$: evict the page under the hand
 $R = 1$: clear R and advance the hand

2 Clock Structure

All page frames are arranged in a circular list.



Meaning of the Hand

The hand points to the oldest candidate page. On a page fault, the OS starts checking from that position.

3 Algorithm Steps

When a page fault occurs:

1. Inspect the page currently under the clock hand.
2. If $R = 0$, evict that page.
3. Insert the incoming page into the same frame.

4. Advance the hand to the next frame.
5. If $R = 1$, clear R , advance the hand, and continue scanning.

4 Decision Table

R bit	Action
0	The page has not been referenced recently. Evict it.
1	The page was referenced recently. Clear R , give it a second chance, and move the hand forward.

5 Worked Example

Problem

The hand points to page A . The pages have these referenced bits:

Page	A	B	C	D	E
R	1	1	0	1	0

Which page is evicted?

Solution

Checked page	R	Action
A	1	Clear R , advance hand.
B	1	Clear R , advance hand.
C	0	Evict C .

Answer

Page C is evicted because it is the first page found with $R = 0$.

6 Why Clock Is Better Than Moving Lists

Second chance and clock make almost the same replacement decision. The difference is implementation cost.

Algorithm	Structure	Cost issue
Second chance	Linear FIFO list.	Pages may be moved to the end of the list.
Clock	Circular list with a hand.	Pages stay in place; only the hand moves.

Efficiency Point

Clock avoids repeatedly rearranging the page list, so it is a cleaner implementation of second chance.

7 If All Pages Have $R = 1$

Clock still terminates.

1. The hand scans pages one by one.
2. Every page with $R = 1$ has R cleared.
3. After one full rotation, pages previously seen now have $R = 0$.
4. The next suitable page is evicted.

Degeneration

If every page was recently referenced, clock eventually behaves like FIFO because all pages receive a second chance.

8 Final Summary

One-Box Summary

Clock is second chance implemented with a circular list.
The hand scans pages and checks the R bit.
If $R = 0$, evict; if $R = 1$, clear it and advance.
It avoids moving pages around like second chance.

Least Recently Used Page Replacement Algorithm

1 Core Idea

Definition

The **Least Recently Used (LRU)** page replacement algorithm evicts the page that has not been used for the longest time.

LRU is based on locality. If a page was used recently, it is likely to be used again soon. If a page has not been used for a long time, it is a better victim.

LRU Rule

evict the resident page with the oldest last-use time

2 Why LRU Approximates Optimal

The optimal algorithm evicts the page whose next use is farthest in the future. LRU cannot know the future, so it uses the past as an estimate.

Observation	LRU assumption
Recently used pages often stay useful.	Keep recently used pages in memory.
Pages unused for a long time often remain unused.	Evict the page with the oldest last reference.

Main Intuition

LRU replaces impossible future knowledge with practical history: use the past to predict near-future behavior.

3 Linked-List Implementation

A direct LRU implementation keeps all pages in memory in a list.

front = most recently used, rear = least recently used

Event	List action
Page is referenced	Move that page to the front of the list.
Page fault occurs	Evict the page at the rear of the list.

Cost Problem

The list must be updated on every memory reference. Finding the page, removing it, and moving it to the front is too expensive for normal software implementation.

4 Worked Example

Problem

The LRU list is:

Front $[C, A, D, B]$ Rear

Page A is referenced. What happens to the list?

Solution

Page A was just used, so it becomes the most recently used page.

$$[C, A, D, B] \rightarrow [A, C, D, B]$$

Answer

Move A to the front. If a page fault happens next, LRU evicts B , the page at the rear.

5 Counter-Based Hardware Implementation

LRU can also be implemented with special hardware.

Counter Method

Use a hardware counter C that increments after each instruction. Each page table entry stores the value of C when that page is referenced.

The page table entry then contains a timestamp showing the page's most recent use.

Step	Action
After each instruction	Hardware increments counter C .
After each memory reference	Hardware stores current C in the referenced page's page table entry.
On page fault	OS scans the counters and finds the smallest value.
Victim	The page with the smallest counter is least recently used.

Counter Victim Rule

victim = page with minimum stored counter value

6 Counter Example

Problem

The current counter value is 900. The page table stores these last-use counters:

Page	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
Last-use counter	880	410	895	700

Which page is LRU?

Solution

LRU chooses the smallest last-use counter:

$$\min(880, 410, 895, 700) = 410$$

The value 410 belongs to page *B*.

Answer

Page *B* is least recently used, so LRU evicts *B*.

7 Why Exact LRU Is Expensive

Implementation	Cost
Linked list	Must update the list on every memory reference.
Counter method	Needs hardware counter and timestamp field in every page table entry.
Victim selection	OS may need to examine many page-table entries to find the smallest timestamp.

Practical Point

LRU is theoretically realizable, but exact LRU is costly. Many real systems use cheaper approximations instead.

8 Final Summary

One-Box Summary

LRU evicts the page unused for the longest time.
It approximates optimal by using recent past behavior.
Exact LRU needs costly per-reference tracking.
It can be implemented with a list or hardware counters, but both are expensive.

Simulating LRU in Software

1 Core Idea

Problem

Exact LRU needs expensive hardware or per-reference list updates. Software algorithms approximate LRU using periodic page-usage information.

Two software approaches are important:

- **NFU (Not Frequently Used)**: counts how often each page was referenced.
- **Aging**: improves NFU by making old references gradually lose importance.

2 NFU Algorithm

Definition

The **Not Frequently Used (NFU)** algorithm gives each page a software counter. At every clock interrupt, the page's R bit is added to its counter.

Each page counter starts at 0. On every clock tick:

$$\text{counter}[p] \leftarrow \text{counter}[p] + R[p]$$

When a page fault occurs, NFU evicts the page with the lowest counter.

Counter value	Meaning
High	Page has been referenced many times in the past.
Low	Page has been referenced fewer times.

3 Main Problem with NFU

NFU Does Not Forget

NFU keeps adding to counters but does not reduce old counts. A page heavily used long ago can look important forever.

Example: in a multipass compiler, pages used heavily in pass 1 may keep high counters even after pass 1 is finished. Later passes may need different pages, but NFU may still protect the old pass-1 pages.

Problem

NFU measures frequency, but it does not measure recency well.

4 Aging Algorithm

Definition

Aging is a software approximation of LRU that shifts each page's counter right at every clock tick and inserts the current R bit into the leftmost bit.

Aging modifies NFU in two ways:

1. Shift each counter right by 1 bit.
2. Insert the page's current R bit into the leftmost bit.

Aging Update

For an n -bit counter:

$$\text{counter}[p] \leftarrow (R[p] \ll (n - 1)) + (\text{counter}[p] \gg 1)$$

For an 8-bit counter, a referenced page gets a leading 1. A page not referenced gets a leading 0.

5 How the R Bit Changes

Important

The R bit changes because of page references during a clock interval. It is not produced by the aging counter.

For each page:

1. At the start of a clock interval, the OS has cleared R to 0.
2. If the CPU reads or writes that page during the interval, hardware sets $R = 1$.
3. If the page is not touched during the interval, R stays 0.
4. At the next clock tick, the OS reads R , updates the aging counter, then clears R back to 0 for the next interval.

R Bit Life Cycle

OS clears $R \rightarrow$ CPU may set $R \rightarrow$ clock tick samples $R \rightarrow$ OS clears R again

Meaning of the Top Row in the PNG

Each top row, such as 1,0,1,0,1,1, is a record of which pages were touched during that clock interval. It is not calculated from the previous top row.

Six R Values for Six Pages

The figure has six pages, numbered 0 through 5. Therefore each clock tick shows six R values, one for each page.

Page	0	1	2	3	4	5
R bits at one tick	1	0	1	0	1	1

This means:

Page	Meaning
Page 0	$R = 1$, so page 0 was referenced in this interval.
Page 1	$R = 0$, so page 1 was not referenced in this interval.
Page 2	$R = 1$, so page 2 was referenced in this interval.
Page 3	$R = 0$, so page 3 was not referenced in this interval.
Page 4	$R = 1$, so page 4 was referenced in this interval.
Page 5	$R = 1$, so page 5 was referenced in this interval.

No Direct Link Between Consecutive R Rows

The next set of R values is a fresh measurement for the next clock interval. It is not computed from the previous R values. It depends only on which pages the CPU touched during that next interval.

For example:

Page	0	1	2	3	4	5
Tick 0 R	1	0	1	0	1	1
Tick 1 R	1	1	0	0	1	0

Tick 1's row means pages 0, 1, 4 were referenced during the next interval. It does not mean the OS transformed tick 0's row into tick 1's row.

6 Aging Figure

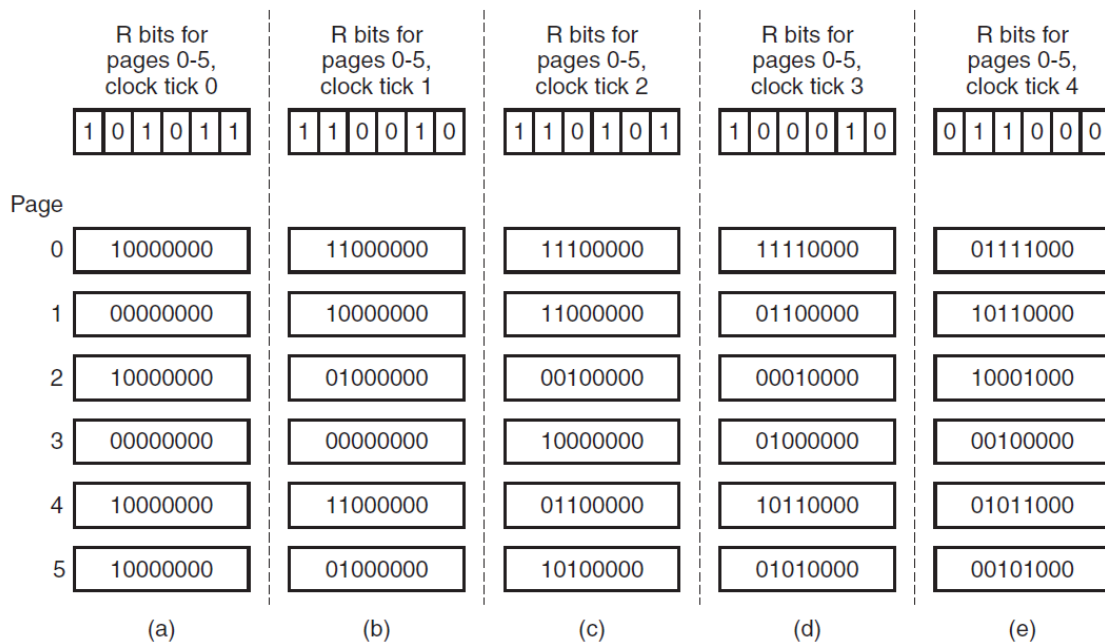


Figure 3-17. The aging algorithm simulates LRU in software. Shown are six pages for five clock ticks. The five clock ticks are represented by (a) to (e).

How to Read the Figure

Each column is one clock tick. The row for each page shows its aging counter after shifting right and inserting that tick's *R* bit on the left.

Detailed Reading

The figure tracks six pages: 0, 1, 2, 3, 4, 5. Each page has an 8-bit aging counter.

Part of figure	Meaning
Top row of each column	The <i>R</i> bits for pages 0 through 5 during that clock interval.
Rows labeled 0 to 5	The aging counters for pages 0 through 5.
Leftmost bit of a counter	Whether the page was referenced in the most recent interval.
Bits farther right	Older reference history. Each tick shifts old history one position to the right.

Column (a): After Clock Tick 0

The *R* bits are:

1, 0, 1, 0, 1, 1

So pages 0, 2, 4, 5 were referenced. Their counters get a leftmost 1:

Page	Counter after tick 0
0	10000000
1	00000000
2	10000000
3	00000000
4	10000000
5	10000000

Column (b): After Clock Tick 1

Now the R bits are:

1, 1, 0, 0, 1, 0

Before inserting these bits, every old counter shifts right. For example:

page 0 : $10000000 \gg 1 = 01000000$

Since page 0 has $R = 1$, insert 1 at the left:

$01000000 \rightarrow 11000000$

Page 2 has $R = 0$, so its old counter shifts right and gets a leftmost 0:

$10000000 \gg 1 = 01000000$

That is why page 2's counter becomes 01000000.

What the Final Column Means

In column (e), the counters are:

Page	Counter
0	01111000
1	10110000
2	10001000
3	00100000
4	01011000
5	00101000

The smallest counter is the best victim because it shows the weakest recent-use pattern.

Page	Interpretation from column (e)
1 and 2	Large leftmost bits, so they were referenced recently. Poor victims.
0 and 4	Some recent history, but not the smallest counters.
3 and 5	Both have small counters, so they are older candidates.

Victim from the Figure

In column (e), page 3 has counter 00100000, while page 5 has 00101000. Since $00100000 < 00101000$, aging chooses page 3.

Why This Is Not Exact LRU

Pages 3 and 5 were both not referenced for the last two ticks. Aging cannot know the exact order of references inside an earlier clock interval, because it records only one R bit per interval.

7 One Counter Update Example

Problem

An 8-bit aging counter is:

01010000

At the next clock tick, the page has $R = 1$. What is the new counter?

Solution

First shift right:

$$01010000 \gg 1 = 00101000$$

Then insert $R = 1$ into the leftmost bit:

$$00101000 \rightarrow 10101000$$

Answer

The new aging counter is:

10101000

8 Choosing a Victim

When a page fault occurs, aging chooses the page with the lowest counter.

Victim Rule

victim = page with the smallest aging counter

A lower counter means the page has not been referenced recently, or has been referenced less often in recent clock intervals.

9 Why Aging Approximates LRU

Recent references are stored in high-order bits, so they matter more than old references.

Bit position	Meaning
Leftmost bits	Recent clock ticks; most important.
Rightmost bits	Older clock ticks; less important.
All zeros	Page has not been referenced during the tracked history window.

Aging Effect

Old references fade because the counter shifts right on every clock tick.

10 How Aging Differs from True LRU

Aging is close to LRU, but it is not exact.

Difference	Explanation
Only one bit per interval	Aging knows whether a page was referenced during a clock tick, but not exactly when inside that interval.
Finite counter size	With 8 bits, only the last 8 clock intervals are represented. Older history is lost.

Limited Horizon

If two pages both have counter 00000000, aging cannot tell whether one was last used 9 ticks ago and the other 1000 ticks ago.

11 Why 8 Bits Can Be Enough

If the clock tick is about 20 msec and the counter has 8 bits, the tracked window is:

$$8 \times 20 \text{ msec} = 160 \text{ msec}$$

Practical Interpretation

If a page has not been referenced for about 160 msec, it is often a reasonable replacement candidate.

12 NFU vs Aging

Algorithm	Main idea	Main weakness
NFU	Add R to a counter at each clock tick.	Old references remain forever.
Aging	Shift right, then insert R at the leftmost bit.	Approximate only; loses exact order inside clock intervals.

13 Final Summary

One-Box Summary

NFU approximates use frequency with counters.
NFU fails because it does not forget old references.
Aging shifts counters right and inserts R at the left.
Aging approximates LRU by making recent references count more.

Working Set Page Replacement Algorithm

1 Core Idea

Definition

The **working set** of a process is the set of pages the process is currently using.

The working set algorithm tries to keep the current working set in memory. When a page fault occurs, it prefers to evict a page that is **not** in the working set.

Replacement Goal

evict a page not in the current working set

2 Demand Paging

Demand Paging

Demand paging loads a page only when the process actually references it.

At process startup, none of the process's pages may be in RAM. The first instruction fetch causes a page fault. More page faults usually follow for global data, heap pages, and stack pages.

After a short startup period, the process normally has the pages it needs and page faults become less frequent.

3 Locality of Reference

Most programs do not use their whole address space uniformly.

Locality

During one phase of execution, a process usually references a small cluster of pages repeatedly.

Example: one pass of a compiler may use parser code and symbol-table pages. A later pass may use optimization code and different data pages. The working set changes, but usually slowly.

4 Thrashing

Thrashing

A process is **thrashing** when it causes page faults so often that most time is spent handling faults instead of executing useful instructions.

Thrashing happens when available memory is too small to hold the process's working set.

Why Thrashing Is Bad

A normal instruction may take nanoseconds, but reading a page from disk may take milliseconds. If page faults occur every few instructions, the program becomes extremely slow.

5 Working Set Model

Let:

- k = number of most recent memory references considered;
- t = current time;
- $w(k, t)$ = set of pages used by the k most recent memory references at time t .

Working Set

$$w(k, t) = \{\text{pages referenced in the } k \text{ most recent memory references at time } t\}$$

As k grows, the working set can only stay the same or grow. It cannot shrink, because looking farther back can include more pages.

Monotonic Behavior

$$k_1 < k_2 \quad \Rightarrow \quad w(k_1, t) \subseteq w(k_2, t)$$

6 Prepaging

Prepaging

Prepaging loads a process's expected working-set pages before allowing the process to run.

When a process is swapped out and later resumed, the OS can use its previous working set as a guess about which pages it will need again.

Purpose

Prepaging reduces the burst of page faults that would otherwise happen when a process restarts.

7 Time-Based Working Set

Counting the last k memory references exactly is too expensive. Real systems often approximate the working set using process execution time.

Virtual Time

Virtual time is the amount of CPU time a process has actually used.

Instead of using the last k references, define the working set as pages referenced during the last τ seconds of virtual time.

Time-Based Working Set

working set = {pages referenced during the last τ seconds of virtual time}

8 Working Set Algorithm Figure

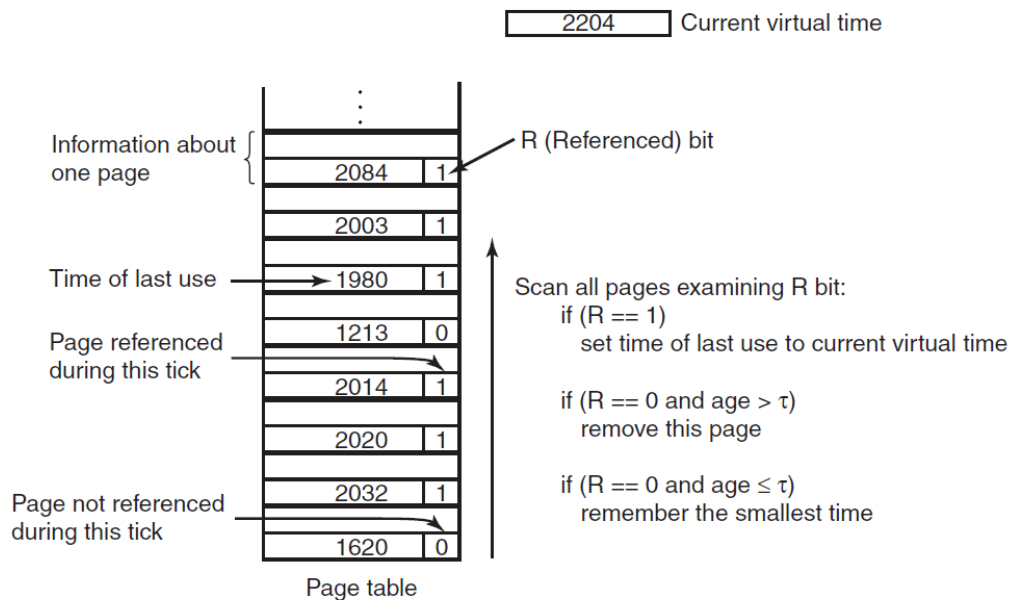


Figure 3-19. The working set algorithm.

What the Figure Shows

The page table stores each page's approximate time of last use and its R bit. Current virtual time is 2204.

9 How to Read the Figure

Field	Meaning
Current virtual time 2204	CPU time already used by this process.
Time of last use <i>R</i> bit	Approximate virtual time when this page was last referenced. 1 means the page was referenced during the current clock tick; 0 means it was not.
Age	Current virtual time minus time of last use.

Page Age

$$\text{age} = \text{current virtual time} - \text{time of last use}$$

10 Algorithm Cases

On a page fault, the OS scans page-table entries for pages that are currently in memory.

Case	Action
$R = 1$	Page was referenced during this tick. Set its time of last use to the current virtual time. Do not evict it.
$R = 0$ and $\text{age} > \tau$	Page is outside the working set. Evict it.
$R = 0$ and $\text{age} \leq \tau$	Page is still inside the working set. Remember it only as a fallback candidate.

11 Worked Example from the Figure

Assume:

$$\text{current virtual time} = 2204, \quad \tau = 150$$

Consider the page with:

$$\text{time of last use} = 1980, \quad R = 1$$

Since $R = 1$, the page was referenced during this clock tick. The OS updates its time of last use:

$$1980 \rightarrow 2204$$

It is kept in memory.

Referenced Page

If $R = 1$, the page is definitely in the working set, so it is not a victim.

Now consider the page with:

$$\text{time of last use} = 1620, \quad R = 0$$

Compute its age:

$$\text{age} = 2204 - 1620 = 584$$

Compare with $\tau = 150$:

$$584 > 150$$

Eviction Candidate

This page has not been used recently enough, so it is outside the working set and can be evicted.

12 If No Page Is Clearly Outside

Sometimes the scan finds no page with:

$$R = 0 \quad \text{and} \quad \text{age} > \tau$$

Then all pages appear to be in the working set.

Situation	Fallback action
Some pages have $R = 0$	Evict the $R = 0$ page with greatest age.
All pages have $R = 1$	Choose a page randomly, preferably a clean page.

13 Why the Exact Definition Is Too Expensive

The exact definition uses the last k memory references. A direct implementation would need to record every memory reference in a large shift register.

Too Expensive

Maintaining and processing the exact last k memory references would be prohibitively expensive, so practical systems use approximations.

14 Strengths and Weaknesses

Point	Explanation
Strength	Reduces page faults by keeping active pages in memory.
Strength	Helps avoid thrashing.
Strength	Supports prepaging when a process resumes.
Weakness	Needs bookkeeping for last-use time and R bits.
Weakness	Choosing τ well is important.

15 Final Summary

One-Box Summary

A working set is the group of pages a process is actively using.
If the working set fits in RAM, page faults stay low.
If it does not fit, the process may thrash.
The algorithm evicts pages not used within the last τ virtual-time units.

WSClock Page Replacement Algorithm

1 Core Idea

Definition

WSClock combines the circular scanning idea of the **clock algorithm** with working-set information such as time of last use, R bit, and M bit.

The basic working set algorithm may scan the whole page table at every page fault. WSClock improves this by keeping only resident page frames in a circular list and scanning from a clock hand.

Main Goal

Find a page that is old enough to be outside the working set and cheap enough to evict.

2 Data Structure

WSClock keeps page frames in a circular list. Each entry stores:

Field	Meaning
Time of last use	Approximate virtual time when this page was last referenced.
R bit	Set when the page was referenced during the current clock interval.
M bit	Set when the page was modified, meaning it is dirty.
Clock hand	Points to the next candidate page frame to inspect.

Age Formula

$$\text{age} = \text{current virtual time} - \text{time of last use}$$

3 WSClock Figure

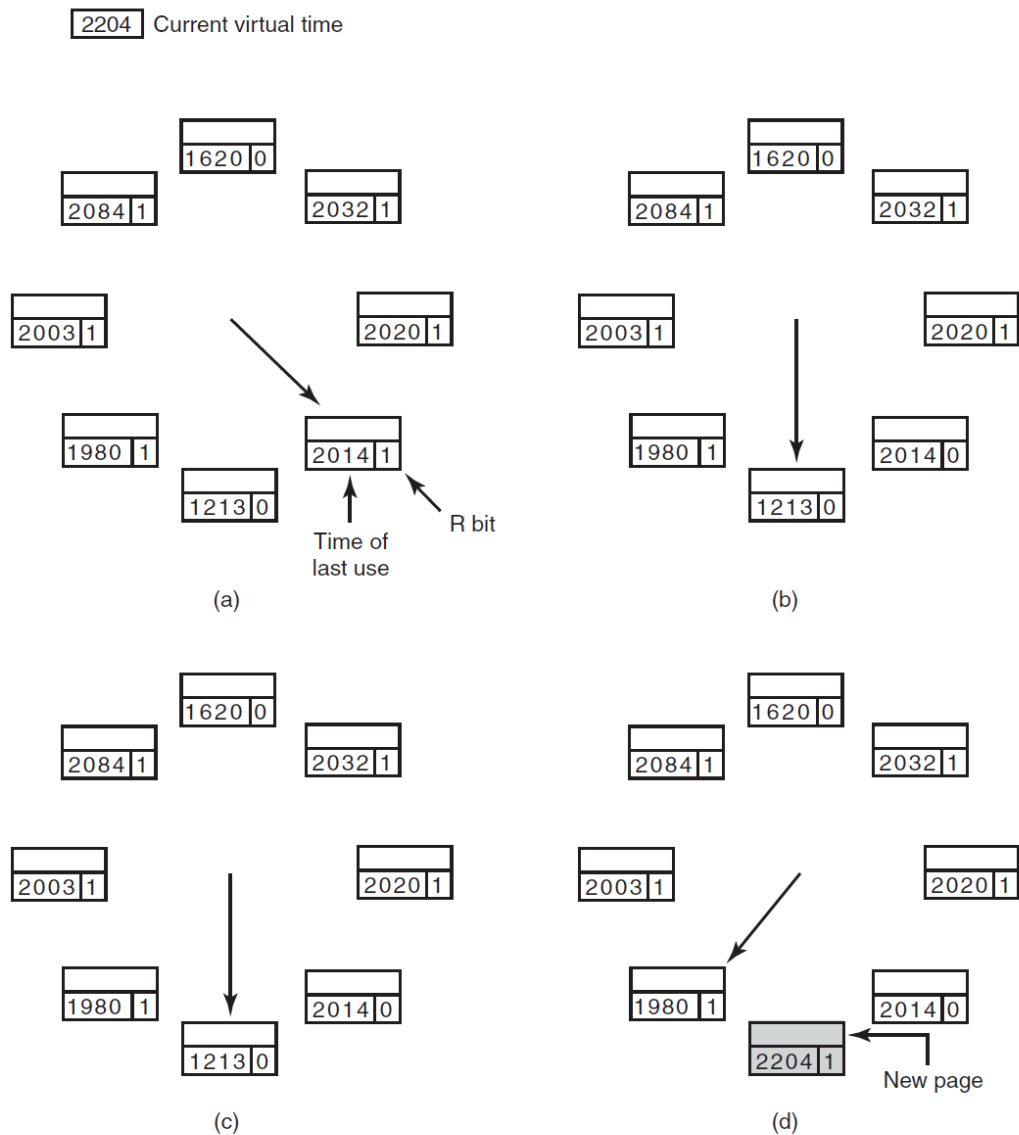


Figure 3-20. Operation of the WSClock algorithm. (a) and (b) give an example of what happens when $R = 1$. (c) and (d) give an example of $R = 0$.

Figure Setup

The current virtual time is 2204. Each small box shows a page frame: the larger number is the time of last use, and the small right-side number is the R bit.

4 Basic Scan Rule

At a page fault, WSClock first examines the page under the hand.

Observed page	Action
$R = 1$	Page was referenced recently. Clear R , update time of last use to current virtual time, advance the hand.
$R = 0$, age $> \tau$, clean	Page is outside the working set and has a valid disk copy. Evict it immediately.
$R = 0$, age $> \tau$, dirty	Schedule disk write, advance the hand, and keep searching.
$R = 0$, age $\leq \tau$	Page is still in the working set. Keep it and advance the hand.

5 Case 1: $R = 1$

If the hand points to a page with $R = 1$, the page was used during the current clock tick.

1. The page is treated as active.
2. Its R bit is cleared to 0.
3. Its time of last use is updated to the current virtual time.
4. The hand advances to the next page.

Panels (a) and (b)

In the figure, panels (a) and (b) show this case. The hand finds a page with $R = 1$, clears its R bit, and moves forward.

6 Case 2: $R = 0$

If $R = 0$, the page was not referenced during the current clock tick. Now WSClock checks its age.

$$\text{age} = 2204 - \text{time of last use}$$

For the page with time of last use 1213:

$$\text{age} = 2204 - 1213 = 991$$

If τ is smaller than 991, then this page is outside the working set.

7 Clean Old Page

Best Victim

If $R = 0$, $\text{age} > \tau$, and $M = 0$, the page is old and clean. WSClock evicts it immediately.

Clean means the disk copy is already up to date. The incoming page can simply replace the old page in that frame.

Panels (c) and (d)

Panels (c) and (d) show this case. The old page with time 1213 and $R = 0$ is replaced by the new page. The new page receives time 2204.

8 Dirty Old Page

If a page is outside the working set but dirty, it cannot be reused immediately.

Condition	Action
$R = 0$, age $> \tau$, $M = 1$	Schedule a disk write for the page.
After scheduling write	Advance the hand and continue searching for another victim.
Reason	A clean old page may appear soon and can be used immediately.

Disk Traffic Control

In practice, the OS may limit how many dirty pages are scheduled for write-back during one sweep to avoid excessive disk traffic.

9 If the Hand Makes a Full Circle

Sometimes the hand returns to its starting point without finding an immediate clean victim.

Full-circle case	What WSClock does
At least one write was scheduled	Keep scanning. Eventually a scheduled write completes, making some page clean. Evict the first clean old page found.
No writes were scheduled	All pages appear to be in the working set. Use a clean page if one was seen; otherwise choose the current page and write it back if needed.

10 Step-by-Step Algorithm

1. Start at the page frame under the clock hand.
2. If $R = 1$, clear R , update last-use time, advance hand.
3. If $R = 0$, compute age.
4. If $\text{age} \leq \tau$, keep the page and advance hand.
5. If $\text{age} > \tau$ and $M = 0$, evict the page.
6. If $\text{age} > \tau$ and $M = 1$, schedule write-back and advance hand.
7. Continue until a usable victim is found.

11 Why WSClock Is Practical

Advantage	Explanation
Avoids full page-table scan	It scans resident frames using a circular list.
Uses recency information	R and last-use time estimate the working set.
Handles dirty pages sensibly	Dirty old pages are scheduled for write-back instead of blocking immediately.
Good practical behavior	It is simpler than exact working set replacement and performs well.

12 Final Summary

One-Box Summary

WSClock = Clock algorithm + working-set information.
If $R = 1$, clear R and advance the hand.
If $R = 0$ and $\text{age} > \tau$ and clean, evict.
If dirty, schedule write-back and keep searching.

Summary of Page Replacement Algorithms

1 Core Idea

Review

Page replacement algorithms choose which resident page to evict when a page fault occurs and no free frame is available.

The best algorithm would evict a page that will not be needed soon. Real systems must approximate this using available information such as arrival order, R bits, M bits, counters, and working-set age.

2 Main Comparison

Algorithm	Comment
Optimal	Not implementable, but useful as a benchmark.
NRU	Very crude approximation of LRU.
FIFO	Simple, but might throw out important pages.
Second chance	Big improvement over FIFO.
Clock	Realistic implementation of second chance.
LRU	Excellent, but difficult to implement exactly.
NFU	Fairly crude approximation to LRU.
Aging	Efficient algorithm that approximates LRU well.
Working set	Good idea, but somewhat expensive to implement.
WSClock	Good efficient practical algorithm.

3 Algorithm Families

Family	Algorithms	Main idea
Future-based	Optimal	Use exact future references. Best possible, but impossible in real systems.
Bit-based	NRU	Use R and M bits to classify pages.
Queue-based	FIFO, second chance, clock	Track arrival order, then improve it using the R bit.
History-based	LRU, NFU, aging	Use past references to estimate future usefulness.
Working-set-based	Working set, WSClock	Keep active pages in memory and evict pages outside the working set.

4 Short Explanations

4.1 Optimal

Optimal

Evicts the page whose next use is farthest in the future.

It gives the lowest possible number of page faults for a known reference string. It cannot be used in practice because the operating system does not know the future.

4.2 NRU

NRU

Divides pages into four classes using R and M , then chooses randomly from the lowest-numbered nonempty class.

It is simple, but crude. It only has coarse information about whether pages were referenced or modified.

4.3 FIFO

FIFO

Evicts the page that has been in memory the longest.

It is easy to implement using a queue, but age in memory is not the same as usefulness. An old page may still be heavily used.

4.4 Second Chance and Clock

Second Chance / Clock

Second chance improves FIFO by checking the R bit. Clock implements the same idea efficiently using a circular list.

If $R = 1$, the page is spared once and R is cleared. If $R = 0$, it can be evicted.

4.5 LRU

LRU

Evicts the page that has not been used for the longest time.

LRU is a strong approximation to optimal, but exact LRU requires costly per-reference tracking or special hardware.

4.6 NFU and Aging

NFU / Aging

NFU counts references. Aging improves NFU by shifting counters so old references fade.

NFU does not forget old activity, so it can preserve pages that are no longer useful. Aging is much better because recent references matter more than old ones.

4.7 Working Set and WSClock

Working Set / WSClock

The working set is the group of pages a process is actively using. WSClock combines this idea with the clock algorithm.

The working set algorithm reduces page faults and thrashing, but scanning can be expensive. WSClock keeps good performance while being efficient to implement.

5 Best Practical Choices

Most Important Practical Algorithms

Two especially important practical algorithms are:

- **Aging**: based on LRU history.
- **WSClock**: based on working-set behavior.

Both give good paging performance and can be implemented efficiently.

6 Final Summary

One-Box Summary

Optimal is best theoretically, but impossible in practice.
FIFO is simple, but may evict useful pages.
Clock is a practical improvement over FIFO/second chance.
Aging approximates LRU efficiently.
WSClock combines working-set ideas with clock efficiency.

Local versus Global Allocation Policies

1 Core Idea

Problem

When a page fault occurs, the operating system must decide whether the victim page must come from the faulting process or may come from any process.

This is a memory-allocation policy question. It is separate from the replacement algorithm itself.

Policy	Meaning
Local replacement	Replace only a page owned by the faulting process.
Global replacement	Replace any page in memory, even if it belongs to another process.

2 Local versus Global Example

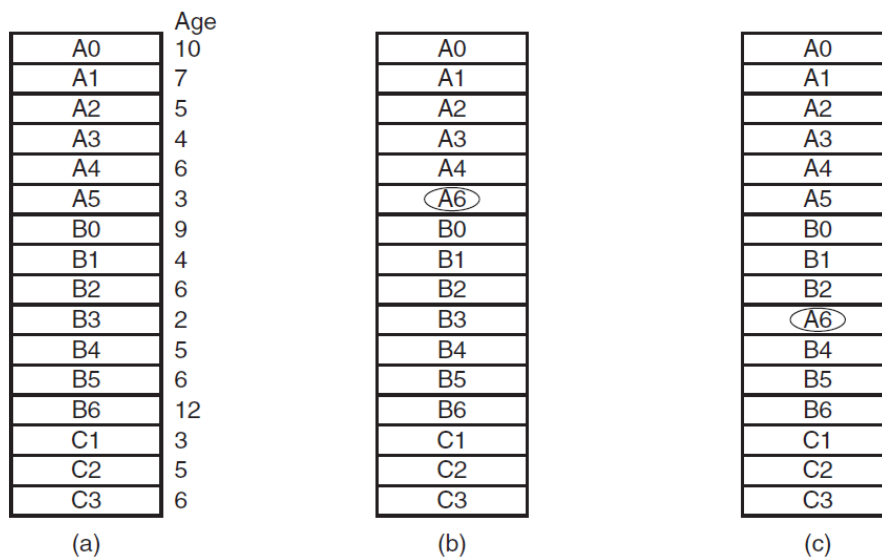


Figure 3-22. Local versus global page replacement. (a) Original configuration. (b) Local page replacement. (c) Global page replacement.

Figure Setup

Processes *A*, *B*, and *C* are runnable. Process *A* gets a page fault and needs to load page *A6*.

In the original configuration, each page has an age value. Lower age means the page was used more recently.

Panel	Meaning
(a)	Original memory configuration with pages from A , B , and C .
(b)	Local replacement: $A6$ replaces one of A 's own pages.
(c)	Global replacement: $A6$ replaces the best victim from all memory.

3 Local Replacement

With local replacement, process A can evict only one of its own pages.

From the figure, A 's pages have ages:

Page	$A0$	$A1$	$A2$	$A3$	$A4$	$A5$
Age	10	7	5	4	6	3

If the algorithm chooses the least recently used page among only A 's pages, it chooses $A5$ because $A5$ has the smallest age value shown for A 's local set.

Local Result

In panel (b), $A6$ replaces $A5$. Other processes keep all their frames.

4 Global Replacement

With global replacement, the victim can belong to any process.

From all pages in the figure, the lowest age is:

$$B3 = 2$$

So the global algorithm chooses $B3$, even though process A caused the page fault.

Global Result

In panel (c), $A6$ replaces $B3$. Process A gains a frame, and process B loses one.

5 Main Difference

Policy	Strength	Weakness
Local	Protects processes from losing frames to others.	Fixed allocation can cause thrashing or waste memory.
Global	Dynamically shifts memory toward processes that need it.	One process can take frames away from another.

General Rule

Global algorithms often work better because process working-set sizes change over time.

6 Why Local Allocation Can Fail

Local replacement effectively gives each process a fixed memory share.

Working-set change	Local-policy problem
Working set grows	The process may thrash even if other processes have spare frames.
Working set shrinks	The process may keep frames it no longer needs, wasting memory.

7 Allocating Frames Among Processes

If global allocation is used, the OS must decide how many frames each process should get.

7.1 Equal Allocation

Divide frames equally among runnable processes.

Example

If 12,416 available frames are shared among 10 processes, each process receives:

$$\left\lfloor \frac{12,416}{10} \right\rfloor = 1241$$

frames, with 6 frames left for a free pool.

Equal allocation is simple, but it ignores process size.

7.2 Proportional Allocation

Allocate frames in proportion to process size.

Why Proportional Allocation Helps

A 300-KB process usually needs more frames than a 10-KB process. Equal allocation gives them the same number, which is often unreasonable.

Even with proportional allocation, each process should receive some minimum number of frames so it can execute basic instructions.

8 Minimum Frame Requirement

Some machines require multiple pages for a single instruction.

Minimum Frames

A single instruction may need pages for the instruction itself, the source operand, and the destination operand. If these cross page boundaries, several pages may be needed at once.

Therefore, allocating too few frames can make a process unable to execute at all.

9 Page Fault Frequency

PFF

Page Fault Frequency (PFF) controls how many frames a process should have by monitoring its page fault rate.

PFF does not choose the victim page directly. It controls the size of the process's allocation set.

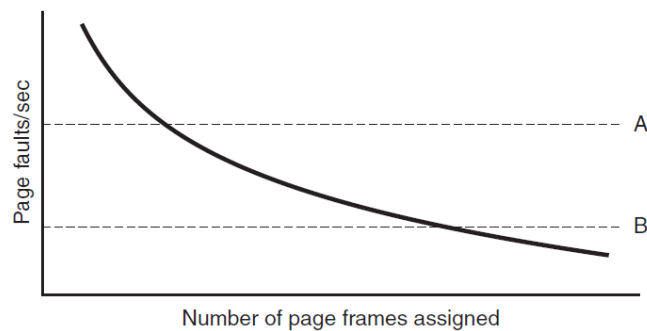


Figure 3-23. Page fault rate as a function of the number of page frames assigned.

10 Reading the PFF Figure

The graph shows that page fault rate usually decreases as a process receives more page frames.

Line	Meaning
A	Fault rate is too high. Give the process more frames.
B	Fault rate is very low. The process may have too many frames. Take some frames away.
Between A and B	Fault rate is acceptable. Keep allocation roughly unchanged.

Simple Running Mean

$$\text{new mean} = \frac{\text{old mean} + \text{faults in last second}}{2}$$

11 Local/Global Is Independent of Some Algorithms

Some replacement algorithms can be used either locally or globally.

Algorithm	Local version	Global version
FIFO	Oldest page owned by faulting process.	Oldest page in all memory.
LRU	Least recently used page owned by faulting process.	Least recently used page in all memory.

12 Algorithms That Are Naturally Local

Working-set-based algorithms refer to a specific process.

Working Set Is Per Process

There is no useful “working set of the whole machine.” The working set concept is tied to one process’s locality of reference.

Therefore, working set and WSClock are naturally local strategies. Taking the union of all processes’ working sets would lose the locality information that makes the method useful.

13 Final Summary

One-Box Summary

Local replacement evicts only from the faulting process.
Global replacement can evict from any process.
Global allocation adapts better to changing working sets.
PFF adjusts frame allocation using page fault rate.
Working-set methods are naturally process-local.

Load Control

1 Core Idea

Definition

Load control is the operating system's decision to reduce the number of processes competing for memory when the system is thrashing.

Even with a good page replacement algorithm and good global frame allocation, the system can still thrash if total memory demand is too high.

Thrashing Condition

$$\sum \text{working sets of runnable processes} > \text{available physical memory}$$

2 Why Replacement Alone Is Not Enough

Page replacement decides which page to evict. Load control decides whether too many processes are active at the same time.

Mechanism	Question answered
Page replacement	Which page should be removed?
Frame allocation	How many frames should each process receive?
Load control	How many processes should compete for memory now?

Key Problem

If every active process needs more memory and no process can safely give memory away, page replacement cannot solve the problem.

3 PFF Symptom

The **Page Fault Frequency (PFF)** algorithm can reveal memory overload.

Thrashing Signal

Thrashing is likely when PFF says some processes need more frames, but no processes have a low enough fault rate to give frames back.

In this situation, giving more frames to one process would hurt another process. The system needs to reduce the active load.

4 Main Solution: Swap Out Processes

Load Reduction

The OS can swap one or more processes to disk, freeing all page frames held by those processes.

The freed frames are then given to the remaining active processes.

1. Detect that the system is thrashing.
2. Pick a process to swap out.
3. Write its pages and process state to disk as needed.
4. Free its page frames.
5. Redistribute those frames among remaining processes.
6. If thrashing continues, swap out another process.

5 Why Swapping Still Matters with Paging

Paging moves individual pages. Swapping removes an entire process from memory.

Technique	What it moves	Purpose here
Paging	Individual pages.	Handle normal page faults.
Swapping	Whole processes or most of their resident pages.	Reduce total memory demand.

Practical Meaning

With virtual memory, swapping is still useful, but mainly as a load-control tool.

6 Connection to Two-Level Scheduling

Load control is similar to two-level scheduling.

Level	Role
Longer-term decision	Which processes stay in memory?
Short-term scheduler	Which in-memory runnable process gets the CPU next?

The OS may periodically swap some processes in and others out, keeping just enough active processes to make the page-fault rate acceptable.

7 Degree of Multiprogramming

Definition

The **degree of multiprogramming** is the number of processes kept active in main memory.

Load control must avoid two extremes.

Extreme	Problem
Too many active processes	Combined working sets exceed memory, causing thrashing.
Too few active processes	CPU may sit idle while processes wait for I/O.

8 Choosing Which Process to Swap Out

The OS should consider more than just process size.

Factor	Why it matters
Process size	Swapping out a large process frees more frames.
Page fault rate	A process causing many faults may be worsening thrashing.
CPU-bound or I/O-bound	Removing too many CPU-bound or I/O-bound processes can unbalance the system.
Remaining process mix	The OS needs enough runnable work to keep the CPU busy.

Tradeoff

Swapping out processes reduces memory pressure, but swapping out too many processes can reduce CPU utilization.

9 Load Control Flow

Decision Flow

high page fault rate
→ PFF shows processes need more frames
→ no process can give frames back
→ swap out one or more processes
→ redistribute freed frames

10 Final Summary

One-Box Summary

Load control handles thrashing caused by too many active processes.
If combined working sets exceed RAM, replacement algorithms are not enough.
The OS can swap out processes to free frames for the rest.
The goal is enough processes for CPU use, but not so many that memory thrashes.

Page Size

1 Core Idea

Definition

Page size is the number of bytes in one virtual page and one physical page frame.

Choosing a page size is a tradeoff. Small pages reduce wasted space, but large pages reduce page-table size, disk-transfer overhead, and TLB pressure.

No Universal Best Size

There is no single optimum page size for all systems. The best choice depends on memory size, TLB size, disk behavior, process sizes, and workload.

2 Hardware and OS Choice

Hardware may support a fixed page size, such as 4096 bytes. The operating system can still treat groups of pages as larger logical pages.

Example

If hardware pages are 4 KB, the OS can treat pairs of consecutive pages as 8-KB units by allocating them together.

Modern systems may support multiple page sizes, for example small pages for user processes and large pages for the kernel or large memory regions.

3 Arguments for Small Pages

3.1 Less Internal Fragmentation

Internal Fragmentation

Internal fragmentation is wasted space inside an allocated page when the segment does not exactly fill the final page.

On average, the last page of a segment is half empty.

If:

- n = number of segments in memory;
- p = page size in bytes;

then expected wasted space is:

Internal Fragmentation

$$\text{wasted space} \approx \frac{np}{2}$$

Smaller p means less average waste in the final page of each segment.

3.2 Less Unneeded Program Memory Loaded

Consider a program with eight sequential phases, each 4 KB.

Page size	Memory needed at one instant
32 KB	Whole 32-KB region may be kept.
16 KB	About 16 KB may be needed.
4 KB or smaller	Only the current 4-KB phase may be needed.

Small-Page Benefit

Smaller pages let memory contain only the currently needed parts of a program.

4 Arguments for Large Pages

4.1 Smaller Page Tables

Small pages mean more pages per process, so page tables become larger.

Page size	Pages needed for a 32-KB program
8 KB	$32/8 = 4$ pages.
512 bytes	$32 \text{ KB}/512 = 64$ pages.

More pages means more page-table entries.

4.2 Better Disk Transfer Efficiency

Disk transfers are usually done one page at a time. Much of the cost comes from seek time and rotational delay, not just the number of bytes transferred.

Case	Approximate load time
64 pages of 512 bytes	$64 \times 10 \text{ msec} = 640 \text{ msec.}$
4 pages of 8 KB	$4 \times 12 \text{ msec} = 48 \text{ msec.}$

Disk-Transfer Point

A small page may take almost as long to transfer as a large page, so many small transfers can be much slower.

4.3 Less TLB Pressure

TLB entries are limited. Larger pages let each TLB entry cover more memory.

Page size	TLB coverage example
4 KB	A 64-KB working set needs at least 16 TLB entries.
2 MB	A 1-MB region can fit under one large-page TLB entry in theory.

TLB Point

Large pages can improve performance because scarce TLB entries cover more address space.

5 Context Switch Cost

On some machines, the OS must load page-table information into hardware registers on each context switch.

Small-Page Cost

Smaller pages create more page-table entries, which can increase page-table loading cost during context switches.

6 Mathematical Tradeoff

Let:

- s = average process size in bytes;
- p = page size in bytes;
- e = bytes required for one page-table entry.

Approximate number of pages per process:

$$\frac{s}{p}$$

Page-table space:

$$\frac{s}{p} \times e = \frac{se}{p}$$

Average internal fragmentation in the final page:

$$\frac{p}{2}$$

Total Overhead

$$\text{overhead}(p) = \frac{se}{p} + \frac{p}{2}$$

7 Finding the Optimum Page Size

To minimize:

$$f(p) = \frac{se}{p} + \frac{p}{2}$$

take the derivative with respect to p :

$$f'(p) = -\frac{se}{p^2} + \frac{1}{2}$$

Set it equal to zero:

$$-\frac{se}{p^2} + \frac{1}{2} = 0$$

Move terms:

$$\frac{1}{2} = \frac{se}{p^2}$$

Multiply by $2p^2$:

$$p^2 = 2se$$

Take square root:

Optimum Page Size

$$p = \sqrt{2se}$$

8 Worked Example

Problem

Find the approximate optimum page size when:

$$s = 1 \text{ MB}, \quad e = 8 \text{ bytes}$$

Solution

Use:

$$p = \sqrt{2se}$$

Assume:

$$1 \text{ MB} = 2^{20} \text{ bytes}$$

Substitute:

$$p = \sqrt{2 \times 2^{20} \times 8}$$

Since $8 = 2^3$:

$$p = \sqrt{2^1 \times 2^{20} \times 2^3}$$

$$p = \sqrt{2^{24}}$$

$$p = 2^{12} = 4096 \text{ bytes}$$

Answer

The optimum page size is approximately:

4 KB

9 Summary of Tradeoffs

Page size	Advantages	Disadvantages
Small pages	Less internal fragmentation; less unused program data in RAM.	Larger page tables; more TLB entries needed; more page transfers.
Large pages	Smaller page tables; better disk transfer efficiency; better TLB coverage.	More internal fragmentation; more unneeded memory may be loaded.

10 Final Summary

One-Box Summary

Small pages reduce internal fragmentation.
 Large pages reduce page-table size, disk overhead, and TLB pressure.
 The simplified overhead model is $\frac{se}{p} + \frac{p}{2}$.
 Its optimum is $p = \sqrt{2se}$.
 For $s = 1 \text{ MB}$ and $e = 8$, the result is 4 KB.

Separate Instruction and Data Spaces

1 Core Idea

Definition

Separate instruction and data spaces means a process has one virtual address space for program instructions and another virtual address space for data.

The instruction space is often called **I-space**. The data space is called **D-space**.

Main Benefit

separate I-space and D-space can double the usable virtual address range

2 Single Address Space

Most computers use one address space for both program text and data.

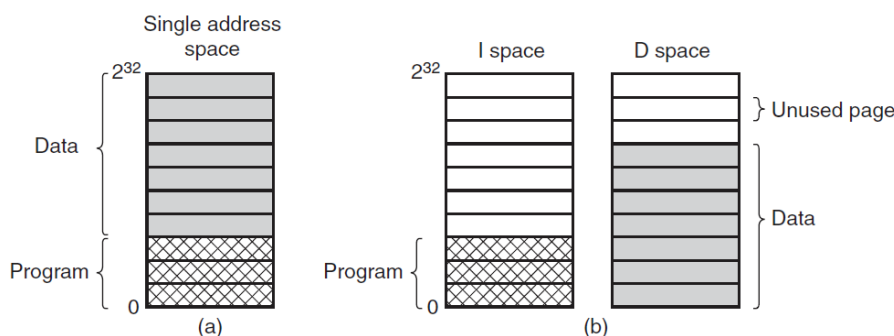


Figure 3-24. (a) One address space. (b) Separate I and D spaces.

In panel (a), the program and data share one address space.

Part	Meaning
Program	Machine instructions, also called program text.
Data	Variables, heap, stack, and other data memory.
Single limit	Program and data together must fit in the same address range.

Problem

If the single address space is small, the program and data compete for the same limited virtual addresses.

3 Separate I-Space and D-Space

Panel (b) shows separate spaces:

- **I-space**: holds instructions;
- **D-space**: holds data.

Each space starts at virtual address 0 and runs up to its own maximum.

Key Difference

Address 0 in I-space is not the same as address 0 in D-space. They belong to different virtual address spaces.

4 Why This Helps

With one address space, program and data must share the same range.

With separate spaces:

program can use a full I-space

data can use a full D-space

Design	Effect
Single address space	Program and data together must fit in one space.
Separate I/D spaces	Program and data each get their own space.

This was especially useful on machines with small address sizes, such as 16-bit systems.

5 Linker Requirement

The linker must know whether separate I-space and D-space are being used.

Why the Linker Cares

In a single address space, data usually starts after the program. With separate I/D spaces, data can be relocated to virtual address 0 in D-space.

So the layout changes:

Mode	Data location
Single address space	Data is placed after program text.
Separate I/D spaces	Data starts at virtual address 0 in D-space.

6 Paging with Separate Spaces

Both I-space and D-space can be paged independently.

Address type	Page table used
Instruction fetch	I-space page table.
Data read or write	D-space page table.

Hardware Role

The hardware knows whether it is fetching an instruction or accessing data, so it can choose the correct page table.

7 OS Implications

Separate I/D spaces do not fundamentally complicate paging.

- I-space has its own page table.
- D-space has its own page table.
- Instruction addresses are translated through the I-space table.
- Data addresses are translated through the D-space table.

The main difference is that the OS tracks two mappings instead of one.

8 Modern Relevance

Modern virtual address spaces are usually large, so separate I/D address spaces are less necessary for ordinary process address-space size.

However, the idea still appears in cache design.

L1 Cache

Many processors split the L1 cache into an instruction cache and a data cache.

This is similar in spirit:

- instruction fetches use the instruction cache;
- data loads/stores use the data cache.

Why Still Useful

Even when virtual address spaces are large, L1 cache space is small and performance-critical, so separating instruction and data traffic can help.

9 Final Summary

One-Box Summary

Single address space: program and data share one virtual range.
Separate I/D spaces: instructions and data have separate ranges.
Instruction fetches use the I-space page table.
Data accesses use the D-space page table.
The idea is still common in split L1 instruction/data caches.

Shared Pages

1 Core Idea

Definition

Shared pages are physical memory pages mapped into the address spaces of two or more processes.

Sharing avoids keeping multiple identical copies of the same program text or library code in RAM.

Main Benefit

If several processes use the same read-only code, one physical copy can serve all of them.

2 What Can Be Shared

Not every page is equally easy to share.

Page type	Sharing difficulty
Program text	Easy to share because it is normally read-only.
Shared libraries	Often sharable when mapped read-only or carefully protected.
Data pages	Harder because processes may write different values.

Key Rule

Read-only pages are naturally sharable. Writable pages need extra mechanisms to avoid one process accidentally changing another process's private data.

3 Sharing with Separate I/D Spaces

If a system supports separate instruction and data spaces, sharing program code is straightforward.

- Processes share the same **I-space** page table for program text.
- Each process has its own **D-space** page table for data.

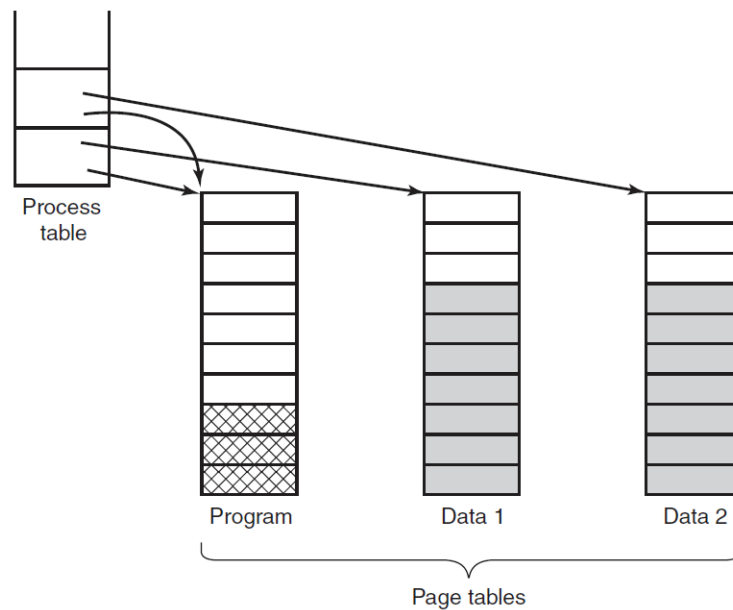


Figure 3-25. Two processes sharing the same program sharing its page tables.

Figure Meaning

Two processes point to the same program page table, but each process points to a different data page table.

4 How the Process Table Uses Page Tables

In this design, page tables are separate data structures, not just fields inside the process table.

Each process table entry can contain:

Pointer	Purpose
I-space page-table pointer	Locates the shared program-text mapping.
D-space page-table pointer	Locates that process's private data mapping.

When the scheduler runs a process, it uses these pointers to set up the MMU with the correct page tables.

5 Why Shared Pages Need Bookkeeping

Sharing creates ownership problems.

Eviction Problem

If process *A* and process *B* share editor code, evicting all of *A*'s pages must not accidentally remove code pages still needed by *B*.

Similarly, when *A* exits, the OS must not free a shared page's disk space if *B* still uses it.

Need for Metadata

The OS needs special data structures, such as reference counts or reverse mappings, to track which pages are shared and who still uses them.

Searching every process's page table to discover whether a page is shared is usually too expensive.

6 Sharing Without Separate I/D Spaces

Sharing is still possible without separate I-space and D-space, but the mechanism is more complicated.

Case	Issue
Shared code	The OS must map the same physical code pages into multiple processes.
Shared data	Writes must be controlled so that private data does not become accidentally shared.

7 Copy on Write

Definition

Copy on write lets processes share data pages until one process tries to modify a shared page.

This is commonly used after the UNIX `fork` system call.

8 Copy-on-Write Flow

After `fork`:

1. Parent and child get separate page tables.
2. Both page tables point to the same physical pages.
3. Shared data pages are marked read-only in both processes.
4. If both only read, no copying occurs.
5. If either process writes to a shared page, a protection fault traps to the OS.
6. The OS copies that page.
7. Each process gets its own private copy.
8. Both copies can then be marked read/write.

Why Copy on Write Is Efficient

Only pages that are actually modified are copied. Pages that are never written, including program text pages, remain shared.

9 Copy-on-Write Example

Scenario

Process P calls `fork`, creating child process C . Both initially share data page X , marked read-only.

Event	Result
P reads X	Allowed. Page remains shared.
C reads X	Allowed. Page remains shared.
P writes X	Trap occurs because X is read-only.
OS handles trap	OS copies X , gives P a private writable copy, and leaves C using the old copy.

10 Summary of Sharing Techniques

Technique	Purpose
Shared read-only code	Avoid duplicate copies of program text or libraries.
Shared I-space page table	Let multiple processes use the same instruction mappings.
Private D-space page table	Keep each process's writable data separate.
Reference tracking	Prevent freeing shared pages still in use.
Copy on write	Delay copying data pages until a write actually happens.

11 Final Summary

One-Box Summary

Shared pages reduce duplicate memory use.
Read-only code is easy to share; writable data is harder.
Separate I/D spaces make code sharing simple.
The OS must track shared pages to avoid freeing them too early.
Copy on write shares data until a process modifies it.

Shared Libraries

1 Core Idea

Definition

Shared libraries are libraries whose code pages can be mapped into multiple processes so that only one physical copy needs to be kept in memory.

On Windows, shared libraries are commonly called **DLLs** or **Dynamic Link Libraries**.

Main Benefit

Shared libraries reduce executable size, disk usage, and RAM usage by avoiding duplicate copies of common library code.

2 Sharing Program Text

If the same program is started twice, most operating systems share its program-text pages automatically.

Page type	Sharing behavior
Text pages	Usually read-only, so they are easy to share.
Data pages	May be private, or shared read-only until modified using copy on write.

Rule

Read-only code can be safely shared. Writable data needs protection or copy on write.

3 Traditional Static Linking

In traditional linking, object files and library functions are copied into the final executable.

```
ld *.o -lc -lm
```

This command links:

- all object files in the current directory;
- the C library;
- the math library.

Undefined External

An **undefined external** is a function or symbol used by a program but not defined in its object files. The linker searches libraries to find it.

Example: if the program calls `printf`, the linker finds `printf` in a library and includes it in the executable. If `printf` calls `write`, then `write` must also be included.

4 Problem with Static Linking

Large modern programs use many libraries, such as graphics, I/O, and user-interface libraries.

Static Linking Waste

If every executable contains its own private copy of common library functions, disk space and RAM are wasted.

For example, if hundreds of programs statically include the same 20-MB to 50-MB graphics library, the same code appears many times on disk and may also appear many times in RAM.

5 How Shared Libraries Work

With shared libraries, the linker does not copy the whole library code into each executable.

Instead, it places small pieces of code called **stubs**.

Stub

A **stub** is a small routine inserted by the linker that connects the program to the real library function at run time.

Depending on the system, a shared library may be loaded:

- when the program starts; or
- when a function from the library is first called.

Paging Detail

The entire shared library is not loaded into RAM at once. It is paged in as needed, so unused functions do not need to occupy memory.

6 Update Advantage

Shared libraries also simplify bug fixes and security updates.

Static library	Shared library
Programs may need recompilation or relinking to get fixes.	Replacing the shared library can update all programs that use it.
Library code is copied into each executable.	Executables bind to the shared copy at run time.

This is important for commercial software where users do not have the source code.

7 Shared Library Address Problem

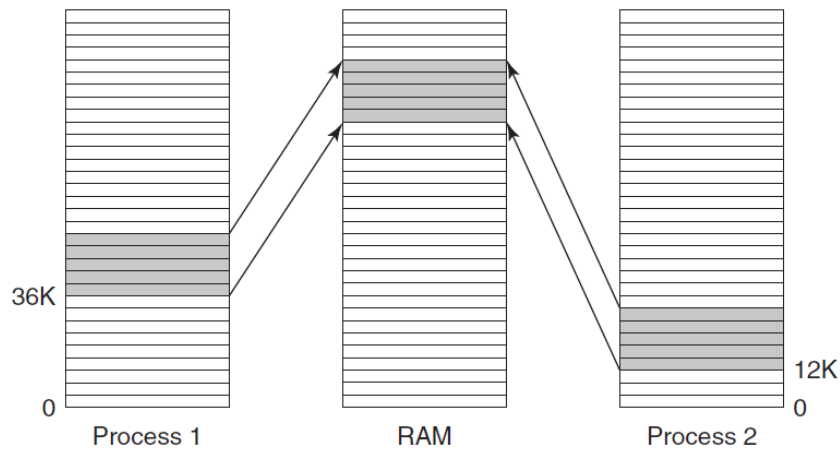


Figure 3-26. A shared library being used by two processes.

Figure Setup

The same 20-KB shared library is mapped into two processes, but at different virtual addresses: 36K in process 1 and 12K in process 2.

The physical RAM location does not matter to the program because the MMU translates virtual addresses to physical frames. The problem is the library's own internal addresses.

8 Why Absolute Addresses Fail

Suppose the first function in the library must jump to byte 16 inside the library.

Process	Correct jump target
Process 1	$36K + 16.$
Process 2	$12K + 16.$

If the library contained an absolute jump address fixed for process 1, it would be wrong for process 2.

Problem

Relocating a shared library separately for each process would create private modified copies of the library pages, defeating the purpose of sharing.

9 Position-Independent Code

Definition

Position-independent code (PIC) is code that uses relative addressing instead of absolute addresses, so it can run correctly no matter where it is mapped.

Instead of saying:

jump to absolute address X

PIC uses instructions like:

jump forward or backward by n bytes

This works because the relative distance between instructions inside the library is the same in every process.

Solution

Shared libraries are compiled to avoid absolute addresses. This lets the same physical code pages be mapped at different virtual addresses in different processes.

10 Final Summary

One-Box Summary

Shared libraries let processes share common library code.
Static linking copies library code into each executable.
Shared linking uses stubs and binds to library code at run time.
Libraries may be mapped at different virtual addresses in different processes.
Position-independent code solves this by using relative addressing.

Mapped Files, Cleaning Policy, and Virtual Memory Interface

1 Core Idea

Three Related Design Issues

This note covers three virtual-memory design issues:

- **memory-mapped files:** files accessed through virtual memory;
- **cleaning policy:** keeping free clean frames available;
- **virtual memory interface:** giving programmers controlled access to memory mappings.

2 Memory-Mapped Files

Definition

A **memory-mapped file** is a file mapped into a process's virtual address space so the process can access the file as if it were memory.

Instead of calling `read` and `write`, the program can treat the file like a large array of bytes.

Main Idea

file on disk $\longleftrightarrow$ region of virtual address space

3 How Mapping Works

The process asks the OS to map a file into part of its virtual address space.

1. The process issues a system call to map the file.
2. The OS creates virtual-memory mappings for the file region.
3. Usually, no file pages are loaded immediately.
4. When the process touches a mapped page, a page fault occurs.
5. The OS demand-pages that part of the file into RAM.
6. Modified mapped pages are written back to the file when unmapped or when the process exits.

Demand Paging

Mapped files are usually loaded page by page, only when touched.

4 Mapped Files as an I/O Model

Mapped files provide an alternative to explicit file I/O.

I/O model	How the program accesses data
Traditional I/O	Calls <code>read</code> and <code>write</code> .
Memory-mapped I/O	Reads and writes memory addresses in the mapped region.

Convenience

For some programs, treating a file as a byte array in memory is simpler than manually issuing reads and writes.

5 Sharing with Mapped Files

If two processes map the same file, they can share memory through it.

Action	Effect
Process <i>A</i> writes to mapped region	The shared memory changes.
Process <i>B</i> reads same mapped region	It sees the data written by <i>A</i> .
Both map a scratch file	The file acts as a shared-memory communication channel.

High-Bandwidth Communication

Mapped files can provide fast interprocess communication because data can be shared through memory instead of copied through pipes or messages.

6 Shared Libraries as Mapped Files

Shared libraries are a special case of mapped files.

Connection

A shared library can be mapped into the virtual address spaces of multiple processes. The library file on disk acts as the backing store.

Only the library pages that are actually used need to be brought into RAM.

7 Cleaning Policy

Definition

A **cleaning policy** keeps enough free, clean page frames available so page faults can be handled quickly.

Paging works best when the OS has free frames ready. If every frame is full and dirty, a page fault may need to wait for a disk write before the needed page can be loaded.

8 Paging Daemon

Paging Daemon

A **paging daemon** is a background process that periodically checks memory and prepares free page frames.

The paging daemon usually sleeps. When too few frames are free, it wakes up and starts selecting pages to evict.

Page state	Daemon action
Clean page	Can be moved to the free-frame pool quickly.
Dirty page	Must be written back to disk before becoming a clean reusable frame.

Why This Helps

It is faster to have clean free frames ready before a page fault than to start cleaning only after memory is urgently needed.

9 Remembering Evicted Pages

When a page is evicted, the OS may remember what was in that frame.

Reclaiming

If an evicted page is needed again before its frame is overwritten, the OS can reclaim it from the free-frame pool instead of reading it again from disk.

This improves performance because some evicted pages may be referenced again soon.

10 Two-Handed Clock Cleaning

One cleaning method uses a two-handed clock.

Clock hand	Role
Front hand	Controlled by the paging daemon. It writes dirty pages back to disk or skips clean pages.
Back hand	Used for page replacement. It is more likely to find clean pages because the front hand has already cleaned ahead.

Two-Handed Clock Idea

front hand cleans pages → back hand finds clean victims more easily

11 Virtual Memory Interface

Definition

A **virtual memory interface** gives programmers some control over the memory map instead of making virtual memory completely transparent.

Most of the time, programs simply see a large virtual address space. In advanced systems, programs can explicitly map, unmap, share, or transfer memory regions.

12 Controlled Shared Memory

If processes can name memory regions, one process can give another process access to the same pages.

Operation	Result
Process <i>A</i> names a memory region	The region can be identified by the OS.
Process <i>A</i> gives the name to process <i>B</i>	<i>B</i> can map the same region.
Both access the region	They communicate through shared memory.

13 Message Passing by Page Remapping

Normally, message passing copies data from one address space to another. Page remapping can avoid copying the actual data.

1. Sender places the message in one or more pages.
2. Sender unmaps those pages.
3. Receiver maps those pages into its own address space.
4. Only page mapping information is transferred, not the full message bytes.

Performance Benefit

For large messages, remapping pages can be much faster than copying all the data.

14 Distributed Shared Memory

Definition

Distributed shared memory lets processes on different machines share pages over a network.

The system tries to make remote pages appear like shared memory.

1. A process references a page not currently mapped locally.
2. A page fault occurs.
3. The page fault handler locates the machine holding the page.
4. That machine unmaps or transfers ownership of the page.
5. The page is sent over the network.
6. The receiving machine maps it in.
7. The faulting instruction restarts.

Cost

Distributed shared memory hides communication behind page faults, but network transfer is much slower than local memory access.

15 Final Summary

One-Box Summary

Mapped files map disk files into virtual memory.
They support convenient I/O and shared-memory communication.
Cleaning policy keeps clean free frames ready for page faults.
A paging daemon can clean pages in the background.
Advanced VM interfaces let programs share or remap pages directly.

Operating System Involvement with Paging

1 Core Idea

Main Idea

Paging is supported by hardware, but the operating system still has important work to do at process creation, execution, page fault handling, and process termination.

The MMU performs address translation during normal execution. The OS sets up the structures that make this translation possible and handles exceptional cases such as page faults.

2 Four Times the OS Is Involved

Time	OS paging work
Process creation	Create and initialize paging data structures.
Process execution	Make the process's page table current and reset MMU/TLB state.
Page fault	Locate the missing page, find a frame, load the page, and restart the instruction.
Process termination	Release page tables, physical pages, and disk backing space.

3 Process Creation Time

When a new process is created, the OS prepares its virtual-memory environment.

1. Determine the initial size of the program text and data.
2. Allocate memory for the process's page table.
3. Initialize the page-table entries.
4. Allocate swap-space or backing-store space on disk.
5. Initialize the backing store with program text and data if needed.
6. Record page-table and swap-area information in the process table.

Page Table Residency

The page table does not have to stay in RAM while the process is swapped out. It must be in memory when the process is running.

4 Executable File as Backing Store

Some systems do not copy program text into the swap area at process creation time.

Optimization

Program text can be paged directly from the executable file.

This saves:

- disk space, because the text pages do not need a duplicate swap copy;
- initialization time, because the OS does not need to copy them into swap space first.

5 Process Execution Time

When the scheduler chooses a process to run, the OS must make that process's address space active.

Action	Purpose
Set page-table register or pointer	Tell the MMU which page table belongs to the new process.
Flush the TLB	Remove cached translations from the previous process.
Reset MMU state	Ensure translations use the new process's mappings.
Optional prepaging	Bring likely-needed pages into memory before execution continues.

Why Flush the TLB?

The TLB may contain virtual-to-physical translations from the previously running process. If they are reused incorrectly, the new process could access the wrong physical frames.

6 Optional Prepaging

The OS may bring in some pages before the process actually faults on them.

Example

The page containing the instruction pointed to by the program counter will certainly be needed, so the OS may load it early.

Prepaging can reduce the initial burst of page faults when a process begins or resumes.

7 Page Fault Time

When a page fault occurs, the OS handles the missing-page event.

1. Read hardware registers to find the faulting virtual address.
2. Determine which virtual page contains that address.
3. Locate the needed page on disk.

4. Find a free page frame.
5. If no free frame exists, choose a victim page to evict.
6. If the victim is dirty, write it back to disk.
7. Read the needed page into the selected frame.
8. Update the page table and relevant TLB information.
9. Back up or reset the program counter to the faulting instruction.
10. Restart the instruction.

Page Fault Flow

faulting address → needed page → free frame → load page → restart instruction

8 Why the Instruction Is Restarted

The faulting instruction did not complete successfully because the page was missing.

Restart Rule

After the page has been loaded, the CPU must execute the same instruction again, now with the needed page present.

This is why the OS may need to back up the program counter or otherwise restore the faulting instruction's state.

9 Process Termination Time

When a process exits, the OS must clean up its paging resources.

Resource	Cleanup action
Page table	Release the memory used for the process's page table.
Physical pages	Free the frames owned only by this process.
Swap space	Release backing-store disk space.
Shared pages	Release only if no other process still uses them.

Shared Page Warning

If a page is shared, the OS must not free its memory or disk backing store until the last process using it has terminated or unmapped it.

10 Lifecycle Summary

Stage	Main OS job	Key data structures
Creation	Build address-space metadata.	Page table, process table, swap map.
Execution	Activate correct mappings.	MMU registers, TLB, page-table pointer.
Page fault	Bring missing page into RAM.	Page table, frame table, disk backing store.
Termination	Free resources safely.	Page table, frame table, swap map, shared-page metadata.

11 Final Summary

One-Box Summary

At creation, the OS builds page tables and backing-store metadata.
At execution, it loads the MMU state and flushes stale TLB entries.
At a page fault, it finds, loads, maps, and retries the missing page.
At termination, it releases page tables, frames, and disk space safely.

Page Fault Handling

1 Core Idea

Definition

Page fault handling is the sequence of hardware and operating-system actions that loads a missing page into memory and restarts the faulting instruction.

A page fault is not just a disk read. It involves saving CPU state, validating the address, finding or freeing a frame, doing disk I/O, updating tables, and restarting execution.

Big Picture

trap → identify page → validate access → get frame → load page → restart instruction

2 Overview of the 10 Steps

Step	Main action
1	Hardware traps to the kernel and saves fault context.
2	Assembly routine saves registers and calls the OS.
3	OS identifies the faulting virtual page.
4	OS checks validity and protection, then finds or selects a frame.
5	If victim frame is dirty, schedule disk write and suspend the faulting process.
6	Read the needed page from disk into the clean frame.
7	Disk interrupt arrives; OS updates page tables and frame state.
8	Faulting instruction state is restored.
9	Faulting process is scheduled again.
10	Registers are reloaded and execution resumes in user space.

3 Step 1: Hardware Trap

The page fault begins in hardware.

Trap

The hardware traps to the kernel and saves the program counter. Many machines also save information about the current instruction in special CPU registers.

This saved information lets the OS later restart the instruction as if the page fault had not interrupted it.

4 Step 2: Save Volatile State

An assembly-language routine runs first.

Low-Level Save

The routine saves general registers and volatile CPU state so the operating system does not destroy the user process's execution context.

After saving state, the assembly routine calls the main operating-system page fault handler.

5 Step 3: Identify the Needed Page

The OS must determine which virtual address caused the fault.

Case	OS action
Hardware register gives address	Read the faulting virtual address directly.
No direct address register	Use the saved program counter, fetch the instruction, and decode it in software.

Hard Case

If hardware does not report the faulting virtual address, the OS may need to parse the faulting instruction to discover what memory reference caused the fault.

6 Step 4: Validate Address and Protection

Once the virtual address is known, the OS checks whether the access is legal.

Check	Result
Invalid virtual address	Send a signal or kill the process.
Protection violation	Send a signal or kill the process.
Valid page fault	Continue handling by finding a page frame.

If no free frame exists, the page replacement algorithm is run to choose a victim.

7 Step 5: Dirty Victim Handling

If the selected victim frame contains a dirty page, the page must be written back to disk.

Dirty Victim

A dirty page cannot simply be overwritten because disk does not yet contain its latest contents.

The OS schedules a disk write and marks the frame as busy. The faulting process is suspended, and another process may run while the disk operation completes.

Busy Frame

The selected frame is marked busy so another process does not try to use it before the write or read operation finishes.

8 Step 6: Read the Needed Page

Once the frame is clean, the OS locates the needed page on disk and schedules a disk read.

1. Find the disk address of the needed page.
2. Start disk I/O to read the page.
3. Keep the faulting process suspended during the read.
4. Run another ready process if one is available.

9 Step 7: Disk Interrupt and Table Update

When the disk finishes reading the page, it interrupts the CPU.

Page Arrived

The OS updates the page table so the faulting virtual page now maps to the page frame containing the loaded page.

The frame is marked as normal or available for ordinary use, not busy.

10 Step 8: Restore Faulting Instruction State

The faulting instruction must be retried from the beginning.

Instruction Restart

The OS backs up or resets the program counter so it points to the instruction that caused the page fault.

This is necessary because the memory reference failed before the instruction completed.

11 Steps 9 and 10: Resume the Process

After the page table has been fixed and the instruction state restored:

1. The faulting process is made runnable again.
2. The scheduler eventually chooses it.
3. The OS returns to the assembly routine that saved the registers.
4. That routine reloads registers and other saved state.
5. Control returns to user mode.

Result

The process continues as though the page fault never happened, except for the delay.

12 Clean Flow Summary

Phase	Key work
Trap and save	Preserve program counter, instruction state, and registers.
Diagnose	Determine the faulting address and virtual page.
Validate	Reject illegal addresses or protection violations.
Make room	Find free frame or evict a victim.
Disk I/O	Write dirty victim if needed; read missing page.
Repair mappings	Update page table and frame state.
Restart	Restore state and retry the faulting instruction.

13 Final Summary

One-Box Summary

A page fault traps from user mode into the kernel.
The OS finds the faulting page and checks legality.
It obtains a clean frame, reading/writing disk if needed.
It updates page tables and restarts the same instruction.
Correct restart makes the fault invisible except for time cost.

Instruction Backup

1 Core Idea

Definition

Instruction backup is the problem of restoring the CPU state so the instruction that caused a page fault can be restarted correctly.

When a page fault occurs, the faulting instruction may have executed only partly. After the OS loads the missing page, the same instruction must be executed again from its original starting state.

Main Difficulty

Restarting an instruction is hard if the CPU cannot clearly tell the OS where the instruction began and which side effects have already happened.

2 Why Restart Is Needed

A page fault stops an instruction before it finishes. The OS handles the fault, loads the missing page, and then the process must continue.

Restart Requirement

after page fault handling, rerun the faulting instruction correctly

Correct restart means:

- the program counter points to the beginning of the instruction;
- registers have the correct values;
- partial side effects are undone if necessary.

3 Example Instruction

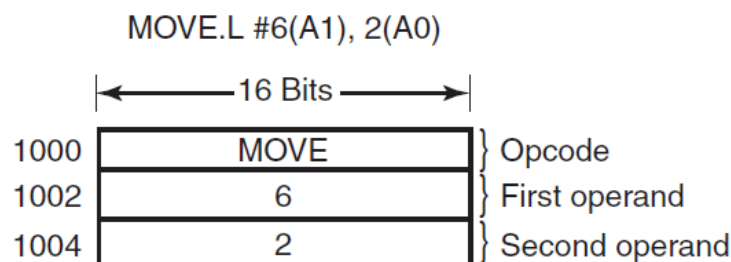


Figure 3-27. An instruction causing a page fault.

The example instruction is:

```
MOVE.L #6(A1), 2(A0)
```

It occupies 6 bytes:

Address	Contents	Meaning
1000	MOVE	Opcode.
1002	6	First operand offset.
1004	2	Second operand offset.

Figure Meaning

The instruction starts at address 1000, but a fault may occur while fetching or using different words of the instruction.

4 Program Counter Ambiguity

The instruction may make several memory references:

- fetch the opcode at 1000;
- fetch or use the first operand information at 1002;
- fetch or use the second operand information at 1004.

Depending on which reference faults, the program counter at trap time might be:

1000, 1002, or 1004

Ambiguity

If the program counter is 1002, the OS may not know whether address 1002 is the middle of an instruction that began at 1000, or the start of a different instruction.

Without extra hardware information, the OS may be unable to unambiguously find the first byte of the faulting instruction.

5 Why Instruction Decoding Can Be Hard

Some CPUs have variable-length instructions. An instruction may contain:

- opcode bytes;
- operand specifiers;
- address offsets;
- immediate constants.

So a word in memory can mean different things depending on where instruction decoding began.

Key Point

The OS cannot safely restart an instruction unless it knows the exact original instruction boundary.

6 Autoincrement and Autodecrement Problems

Some addressing modes modify registers as a side effect.

Autoincrement

Autoincrement means a register is automatically increased as part of executing an instruction.

If a page fault occurs during such an instruction, the OS must know whether the increment already happened.

Case	OS requirement
Register was incremented before the fault	OS may need to decrement it before restarting.
Register was not incremented before the fault	OS must not undo anything.

Autodecrement creates the same kind of problem in the opposite direction.

CPU-Dependent Detail

Whether increment/decrement happens before or after the memory reference can depend on the instruction and CPU model.

7 Hardware Support for Restart

Some CPU designers solve this problem by saving extra internal state.

Hardware information	Why it helps
Hidden saved instruction-start PC	Tells the OS where the faulting instruction really began.
Autoincrement/autodecrement record	Tells which registers were already changed and by how much.
Instruction state registers	Help restore the CPU to a restartable state.

Good Design

If the hardware records enough restart information, the OS can undo partial effects and restart the instruction cleanly.

8 If Hardware Does Not Help

Without hardware support, the OS must reconstruct what happened.

This may require:

- reading the saved program counter;
- decoding nearby instruction bytes;
- determining which operand faulted;
- checking whether registers were already modified;
- undoing side effects manually.

Implementation Burden

If the CPU does not provide enough fault-state information, page fault recovery becomes much more complex for the operating system.

9 Final Summary

One-Box Summary

A page fault can stop an instruction halfway through.
The OS must restart the instruction from its beginning.
Variable-length instructions make the starting address ambiguous.
Autoincrement/autodecrement can leave partial register changes.
Good hardware saves enough state for clean instruction restart.

Locking Pages in Memory

1 Core Idea

Definition

Locking a page in memory, also called **pinning**, means marking a page frame so the page replacement algorithm is not allowed to evict it.

Virtual memory does not remove the need for I/O. When I/O devices read or write data, they often operate directly on physical memory. This creates a problem: paging may try to move or reuse a page while an I/O device is still using it.

Main Rule

A page involved in an active I/O operation must stay in memory until that I/O operation finishes.

2 Where the Problem Comes From

Suppose a process issues a system call such as:

```
read(file, user_buffer, size)
```

The buffer belongs to the process' virtual address space. The operating system translates that buffer to one or more physical page frames so the device can place the incoming data there.

Step	What happens
1	Process starts an I/O request into a user buffer.
2	The process blocks while the device is working.
3	Another process runs and may cause a page fault.
4	A global page replacement algorithm chooses some page frame to evict.
5	If it chooses the buffer page, the active I/O transfer is corrupted.

3 DMA Hazard

DMA

DMA means Direct Memory Access. A device controller transfers data directly to or from physical memory without the CPU copying every byte.

DMA uses physical addresses. Once the I/O operation begins, the device expects the target physical frame to remain the same.

What Can Go Wrong

If the buffer page is evicted during DMA, the physical frame may be reused for another page. The device may then write part of the data into the intended buffer and the rest over an unrelated page.

The bug is especially dangerous because the process that gets damaged may not be the process that requested the I/O.

4 Why Global Replacement Makes This Possible

With a **local** replacement policy, a process normally loses only its own pages. With a **global** replacement policy, any process' page may be selected as the victim.

Policy	Risk during I/O
Local replacement	Lower risk, because the faulting process usually replaces one of its own pages.
Global replacement	Higher risk, because the victim may be a page that belongs to a blocked process doing I/O.

Global replacement is useful for memory utilization, but it needs extra protection for pages that must not move.

5 Solution 1: Pin the I/O Pages

The operating system can mark pages used in an active I/O operation as pinned.

Pinning Rule

pinned page $\Rightarrow$ not eligible for page replacement

The basic sequence is:

1. Translate the user buffer into physical page frames.
2. Pin those page frames.
3. Start the device I/O operation.
4. Keep the process blocked while I/O is in progress.
5. When the device interrupt reports completion, unpin the pages.

Effect

Pinning makes the page replacement algorithm skip those frames, so the device always writes to the correct physical memory.

6 Solution 2: Use Kernel Buffers

Another approach is to avoid device I/O directly into user pages.

Stage	Action
Device I/O	Device transfers data into a kernel buffer.
After I/O completes	OS copies the data from the kernel buffer to the user buffer.

Kernel buffers are controlled by the operating system, so the OS can keep them resident and avoid exposing user pages directly to DMA.

7 Pinning vs Kernel Buffers

Method	Advantage	Cost
Pin user pages	Avoids an extra copy; good for large transfers.	Pinned pages reduce the number of frames available for replacement.
Kernel buffer	Simpler control; avoids DMA directly into pageable user memory.	Requires copying data between kernel memory and user memory.

8 Design Notes

- Pin pages only for the duration of the I/O operation.
- Too many pinned pages can make memory pressure worse.
- Page replacement code must check the pinned status before selecting a victim.
- Kernel buffers trade CPU copy overhead for simpler and safer I/O handling.

9 Final Summary

One-Box Summary

I/O devices may use physical frames directly.
A page used by active I/O must not be evicted.
Pinning locks the page in memory until I/O completes.
Kernel buffers avoid direct I/O into pageable user pages.

Backing Store

1 Core Idea

Definition

Backing store is the disk space used to hold pages when they are not currently in main memory.

Page replacement decides *which* page leaves memory. Backing-store management decides *where* that page is kept on disk and how the operating system finds it again later.

Main Question

When a page is evicted from RAM, where exactly is its disk copy stored?

2 Swap Area

Many systems reserve a special disk region called a **swap partition** or **swap area**.

Feature	Meaning
No normal file system	The OS can use raw block numbers directly.
Lower overhead	No need to translate file offsets into disk blocks.
Separate disk possible	Paging I/O can be separated from file-system I/O.
Free-list management	Free swap chunks are tracked and reused as processes exit.

At boot time, the swap partition is empty. The OS can represent it as one large free region with a starting block and a size.

3 Static Swap Allocation

In the simplest design, each process gets a fixed swap area large enough for its whole virtual image.

Static Swap Address

$$\text{disk address of page } i = \text{start of process swap area} + i$$

Here i is the virtual page number. If the process swap area starts at disk page 1000, then virtual page 5 is stored at disk page 1005.

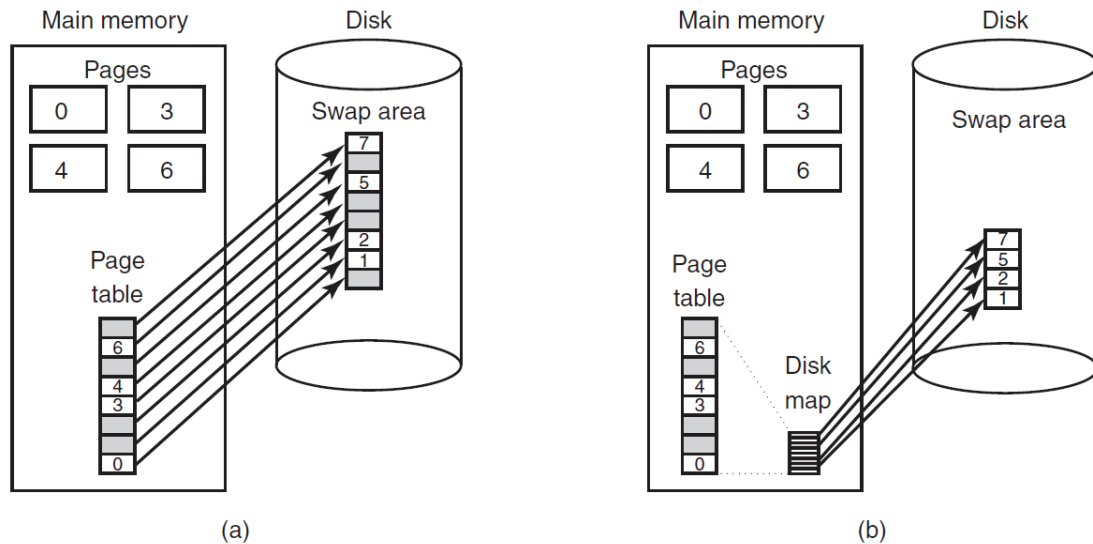


Figure 3-28. (a) Paging to a static swap area. (b) Backing up pages dynamically.

Figure 3-28(a)

In static backing, every virtual page has a fixed disk slot. Pages 0, 3, 4, and 6 are in memory, but the disk area still reserves slots for all pages 0 through 7.

4 How Static Backing Works

State	Disk meaning
Page absent from RAM	Its fixed disk slot is used to bring it back later.
Page present in RAM and clean	The old disk copy is still valid.
Page present in RAM and dirty	The disk copy may be stale and must be overwritten if the page is evicted.

This design makes address calculation simple because the disk location depends only on the process' swap-area start and the virtual page number.

Limitation

Static allocation can waste swap space because a process reserves disk space even for pages that are currently in memory.

5 Growth Problem

Processes may grow after they start:

- program text is usually fixed;

- the data segment can grow, for example because of heap allocation;
- the stack can grow as function calls and local variables accumulate.

If one fixed swap area was reserved at process creation time, it may become too small later. A better design may reserve separate backing areas for text, data, and stack, or allow each area to be made from multiple disk chunks.

6 Dynamic Backing Allocation

The opposite approach is to allocate no swap space in advance. A disk slot is chosen only when a page is actually swapped out.

Figure 3-28(b)

In dynamic backing, pages 1, 2, 5, and 7 are on disk, but they do not need fixed positions based on their virtual page numbers. A disk map records where each swapped-out page is stored.

Method	Advantage	Cost
Static swap area	Very simple address calculation.	Reserves disk space even when pages are in RAM.
Dynamic backing	Uses swap space only for pages that are actually on disk.	Needs a per-process disk map to remember each page's disk location.

7 Swap Partition vs Swap File

Backing type	Explanation
Swap partition	Dedicated disk partition used only for paging. Common in UNIX-style systems.
Swap file	Large preallocated file inside the normal file system. Common in Windows-style systems.

A swap partition avoids normal file-system overhead. A swap file is easier to create when repartitioning the disk is not possible.

8 Executable Files as Backing Store

Program text pages usually come from an executable file and are normally read-only. This creates an optimization:

- clean program-text pages can be discarded instead of written to swap;
- if needed again, they can be reread from the executable file;
- shared-library text pages can use the same idea.

Why This Saves Space

Read-only code already has a valid copy in the executable file, so the OS does not need to store another copy in the swap area.

9 Final Summary

One-Box Summary

Backing store holds pages that are not in RAM.
Static backing gives each virtual page a fixed disk slot.
Dynamic backing allocates disk slots only when pages are evicted.
Clean code pages can often be reread from executable files.

Separation of Policy and Mechanism

1 Core Idea

Definition

Mechanism is the low-level machinery that makes paging work. **Policy** is the decision-making rule that chooses what should happen.

In memory management, separating policy from mechanism means keeping the basic page-fault machinery in the kernel, while moving many higher-level decisions to a separate user-level component.

Main Principle

The kernel provides the mechanism for mapping pages; an external pager can decide the policy for where pages come from, where they go, and sometimes which pages should be replaced.

2 Three Main Parts

Part	Job	Location
MMU handler	Handles machine-dependent MMU details and installs mappings.	Kernel
Page-fault handler	Detects the fault, identifies the needed page, and coordinates the repair.	Kernel
External pager	Manages backing store and paging policy.	User space

3 Why Split Them?

Paging code can become complex because it mixes hardware details, kernel control flow, disk backing store, and replacement decisions. Splitting these concerns makes the system easier to change.

Concern	Best place for it
Hardware-specific translation	MMU handler.
Trap handling and process restart	Kernel page-fault handler.
Backing-store layout and paging policy	External pager.

4 Page Fault Flow

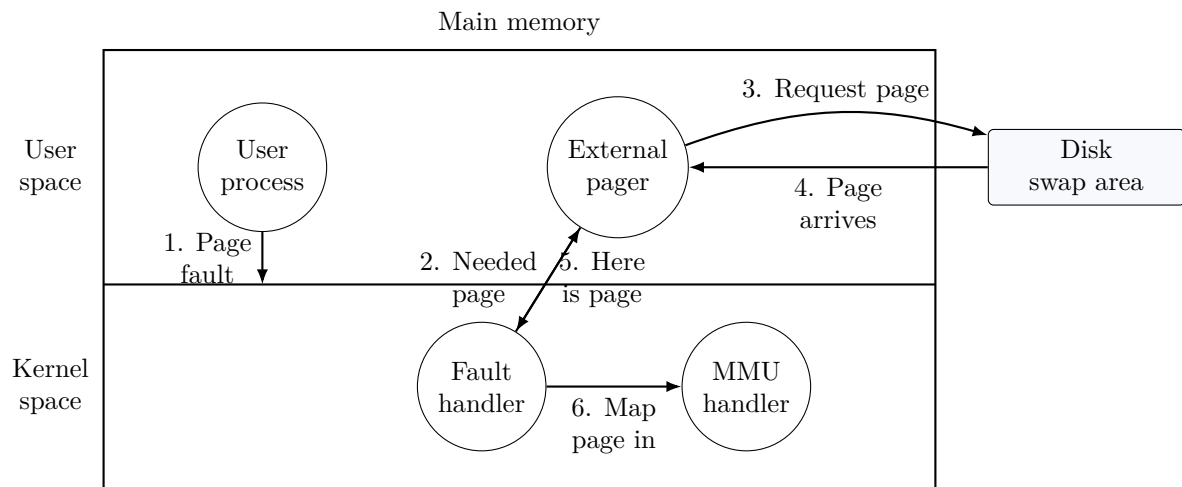


Diagram Meaning

The page fault starts in the kernel, but the needed page may be located and supplied by a user-level external pager.

5 Step-by-Step Explanation

1. The user process references a page that is not currently mapped.
2. The kernel page-fault handler determines which virtual page is needed.
3. The fault handler sends a request to the external pager.
4. The external pager reads the needed page from disk into its own address space.
5. The external pager tells the kernel where the page is.
6. The kernel asks the MMU handler to map that page into the faulting process at the correct virtual address.

After the mapping is installed, the faulting instruction can be restarted.

6 Process Creation and Mapping

When a process starts, the external pager may be notified so it can:

- create or initialize the process page map;
- allocate backing-store space if needed;
- handle future mappings of files, objects, or memory regions.

This keeps higher-level memory decisions outside the low-level kernel mechanism.

7 Where Page Replacement Fits

The clean design would put page replacement policy in the external pager. In practice, this is difficult because many replacement algorithms need hardware-maintained page bits.

Problem	Explanation
Referenced bit R	Shows whether a page was recently read or written.
Modified bit M	Shows whether the page is dirty and must be written back.
External pager limitation	A user-level pager may not directly see all R and M bits.

Because of this, the replacement algorithm may remain in the kernel, while the external pager handles backing-store operations.

8 Advantages and Disadvantages

Side	Explanation
Advantage	More modular code; easier to change paging policy; better separation between hardware details and high-level decisions.
Disadvantage	Extra messages and user-kernel crossings add overhead during page-fault handling.

Tradeoff

This design improves flexibility and modularity, but page faults may become slower because more components communicate during fault handling.

9 Final Summary

One-Box Summary

Mechanism: kernel code that traps, maps, and restarts.
Policy: decisions about paging and backing store.
External pager: user-level process that can manage policy.
Main tradeoff: modularity and flexibility versus extra overhead.

Segmentation

1 Core Idea

Definition

Segmentation is a memory-management scheme in which a process has multiple independent logical address spaces called **segments**.

With pure paging, the virtual address space is one long sequence of addresses:

0, 1, 2, ..., maximum address

This is called a **one-dimensional address space**. Segmentation changes the model. Instead of one long address space, a program can have separate address spaces for different logical parts of the program.

Segmented Address

$$\text{address} = (\text{segment number}, \text{offset inside that segment})$$

2 Why One Address Space Is Limiting

A compiler may build several large tables while compiling a program:

- source text;
- symbol table;
- constants table;
- parse tree;
- call stack.

In a one-dimensional virtual address space, all these structures must be placed in one linear layout.

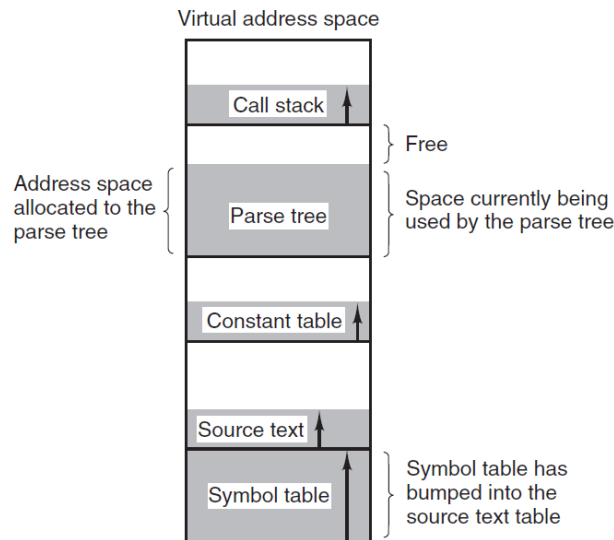


Figure 3-30. In a one-dimensional address space with growing tables, one table may bump into another.

Problem in the Figure

The symbol table has grown until it bumps into the source-text area. Even if other parts of the address space still have free space, this specific region is full.

3 The Need for Independent Growth

Different compiler tables grow differently:

Structure	Growth behavior
Source text	Usually grows as input is saved.
Symbol table	Grows as new names and attributes are discovered.
Constants table	Grows as constants appear in the program.
Parse tree	Grows as syntax is analyzed.
Call stack	Grows and shrinks during procedure calls.

The problem is not that virtual memory is too small in total. The problem is that each table is trapped inside a prechosen region of one linear address space.

4 Segmented Memory

Segmentation gives each logical structure its own segment.

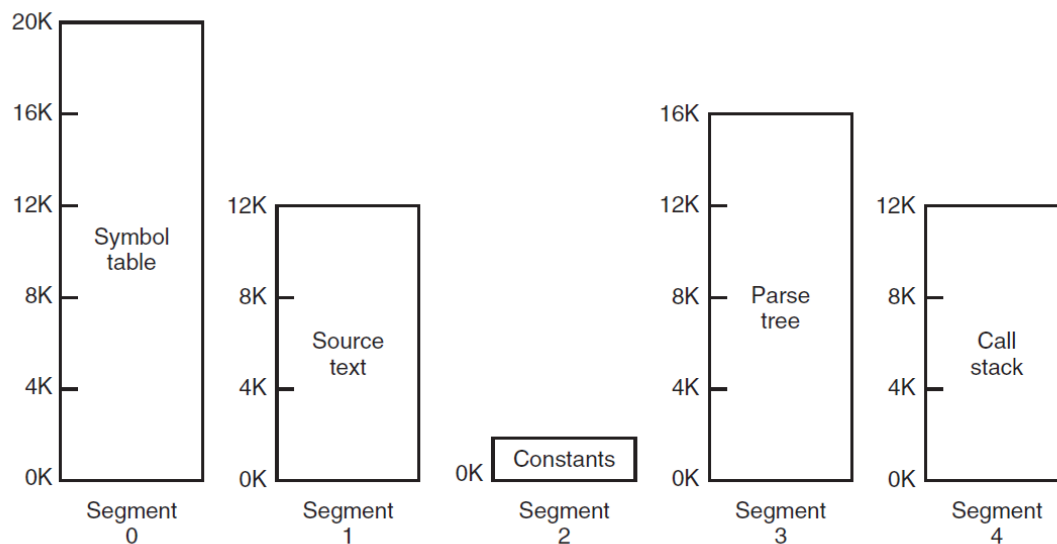


Figure 3-31. A segmented memory allows each table to grow or shrink independently of the other tables.

Figure Meaning

Each compiler table is placed in a separate segment. The symbol table, source text, constants, parse tree, and call stack can grow or shrink independently.

Each segment is a separate linear address space starting at address 0:

Segment 0 : $0 \rightarrow \text{limit}_0$

Segment 1 : $0 \rightarrow \text{limit}_1$

Segment 2 : $0 \rightarrow \text{limit}_2$

The limits may be different because segments do not have to be the same size.

5 Logical Meaning of a Segment

A segment is not just a fixed-size block. It is a logical unit that the programmer or compiler can understand.

Segment may contain	Example
Procedure	Code for one function or module.
Array	A large table or matrix.
Stack	Procedure-call frames and local variables.
Data structure	Symbol table, parse tree, or constants table.

Key Difference

Paging divides memory into equal-size physical units. Segmentation divides a program into meaningful logical units.

6 Address Translation Idea

In segmentation, an address has two parts:

$$(s, d)$$

where:

- s is the segment number;
- d is the offset inside that segment.

The hardware checks whether d is inside the segment limit. If it is valid, the physical address can be formed using the segment base.

Basic Segmentation Translation

$$\text{physical address} = \text{base}[s] + d$$

$$d < \text{limit}[s]$$

If d is greater than or equal to the segment limit, the access is invalid and the OS traps the process.

7 Advantages of Segmentation

1. Independent Growth

Each segment can grow or shrink without colliding with unrelated structures.

Example

If the symbol table grows unusually large, it expands inside its own segment instead of overwriting the source-text table.

2. Easier Linking

If each procedure is placed in its own segment, a call to a procedure can use:

$$(\text{segment } n, 0)$$

The entry point remains offset 0 inside that segment. If the procedure is recompiled and becomes larger, other procedures do not need new starting addresses.

3. Easier Sharing

Segments can be shared between processes. For example, a graphical library can be placed in one segment and mapped into many processes.

4. Better Protection

Different segments can have different permissions.

Segment type	Possible protection
Procedure code	Execute only or read/execute.
Constant table	Read only.
Array or heap data	Read/write, but not execute.
Stack	Read/write, but not execute.

This helps catch bugs such as jumping into data or writing into code.

8 Paging vs Segmentation

Feature	Paging	Segmentation
Programmer view	Usually invisible.	Visible logical units.
Address form	One-dimensional virtual address.	Two-part address: segment and offset.
Unit size	Fixed-size pages.	Variable-size segments.
Main purpose	Efficient physical memory management.	Logical organization, growth, sharing, and protection.
Protection	Usually page-level.	Natural per logical object.

9 Final Summary

One-Box Summary

Segmentation gives a process multiple logical address spaces.
A segmented address has the form (s, d) .
Each segment starts at offset 0 and can grow independently.
Segmentation helps with growth, linking, sharing, and protection.